

Catatan Ujian Akhir Semester

IF2221 Strategi Algoritma

Semester 4 Tahun 2025/2026



NARENDRA DHARMA WISTARA M.

13524044

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

3 Juni 2026

Daftar Isi

1	Informasi Ujian	1
1.1	Capaian Pembelajaran Mata Kuliah (CPMK)	1
1.2	Tabel Hubungan CPMK dan Materi	1
2	Daftar Materi yang Diujikan	1
3	Outline Keseluruhan Materi	1
3.1	Backtracking	2
3.2	Branch and Bound	2
3.3	Route Planning	2
3.4	String Matching	2
3.5	String Matching with Regex	2
3.6	Program Dinamis	2
3.7	Teori P, NP, NPC	2
4	Backtracking	3
4.1	Outline Fundamental Konsep Materi	3
4.2	Pentingnya Materi dan Relevansinya	3
4.2.1	Mengapa Materi Ini Penting	3
4.2.2	Implementasi Dasar	4
4.2.3	Relevansi dengan Industri Saat Ini	4
4.2.4	Dependensi dan Hubungan dengan Materi Lain	4
4.3	Pola Soal UAS Sebelumnya dan Prioritas	4
4.3.1	Formula Eksplisit yang Perlu dikuasai	5
4.4	Penjelasan Konsep Fundamental	5
4.4.1	Backtracking sebagai Perbaikan Exhaustive Search	5
4.4.2	Properti Umum Algoritma Runut-balik	6
4.4.3	Pohon Ruang Status	6
4.4.4	Aplikasi Backtracking	6
4.5	Komponen Kunci Penting	7
4.6	Algoritma dan Rumus Penting	7
4.6.1	Skema Umum Backtracking Rekursif	7
4.6.2	Fungsi Pembatas Sum of Subsets	7
4.7	Checklist Pemahaman	8
4.7.1	Submateri yang Telah Dibahas	8
4.7.2	Prioritas Belajar	8
5	Branch and Bound	8
5.1	Outline Fundamental Konsep Materi	8
5.2	Pentingnya Materi dan Relevansinya	9
5.2.1	Mengapa Materi Ini Penting	9
5.2.2	Implementasi Dasar	9
5.2.3	Relevansi dengan Industri Saat Ini	9
5.2.4	Dependensi dan Hubungan dengan Materi Lain	9
5.3	Pola Soal UAS Sebelumnya dan Prioritas	9
5.3.1	Formula Eksplisit yang Perlu dikuasai	10
5.4	Penjelasan Konsep Fundamental	11
5.4.1	Definisi Branch and Bound	11
5.4.2	Pemangkasan pada B&B	11
5.4.3	Aplikasi B&B	11
5.5	Komponen Kunci Penting	11
5.6	Algoritma dan Rumus Penting	12

5.6.1	Cost Simpul	12
5.6.2	Bound TSP dengan Reduced Cost Matrix	12
5.7	Checklist Pemahaman	12
5.7.1	Submateri yang Telah Dibahas	12
5.7.2	Prioritas Belajar	12
6	Route Planning	13
6.1	Outline Fundamental Konsep Materi	13
6.2	Pentingnya Materi dan Relevansinya	14
6.2.1	Dependensi dan Hubungan dengan Materi Lain	14
6.3	Pola Soal UAS Sebelumnya dan Prioritas	15
6.4	Penjelasan Konsep Fundamental	15
6.4.1	Model Graf untuk Route Planning	15
6.4.2	Frontier, Simpul Ekspan, dan Kondisi Berhenti	16
6.4.3	<i>Uninformed Search</i>	16
6.4.4	Uniform Cost Search	17
6.4.5	<i>Informed Search</i> dan Heuristik	17
6.4.6	Greedy Best-First Search	18
6.4.7	A*	18
6.4.8	Admissible dan Consistent Heuristic	18
6.5	Komponen Kunci Penting	19
6.6	Algoritma dan Rumus Penting	19
6.6.1	Biaya Lintasan	19
6.6.2	Fungsi Evaluasi UCS, Greedy Best-First Search, dan A*	20
6.6.3	Admissibility Heuristic	20
6.6.4	Consistency Heuristic	21
6.6.5	Pseudocode Pencarian Best-First Generik	21
6.6.6	Template Tabel Iterasi UAS	22
6.7	Kesalahan Umum dan Edge Case	22
6.8	Checklist Pemahaman	23
6.8.1	Submateri yang Telah Dibahas	23
6.8.2	Prioritas Belajar	23
7	String Matching	23
7.1	Outline Fundamental Konsep Materi	24
7.2	Pentingnya Materi dan Relevansinya	25
7.2.1	Dependensi dan Hubungan dengan Materi Lain	25
7.3	Pola Soal UAS Sebelumnya dan Prioritas	25
7.4	Penjelasan Konsep Fundamental	26
7.4.1	Definisi Teks, Pattern, dan Indeks Kecocokan	26
7.4.2	Prefix, Suffix, dan Border	26
7.4.3	Brute Force	27
7.4.4	Knuth-Morris-Pratt	27
7.4.5	Boyer-Moore	27
7.5	Komponen Kunci Penting	28
7.6	Algoritma dan Rumus Penting	28
7.6.1	Brute Force String Matching	28
7.6.2	Fungsi Pinggiran KMP	29
7.6.3	Pseudocode KMP	29
7.6.4	Last Occurrence Boyer-Moore	30
7.6.5	Pseudocode Boyer-Moore dengan Character-Jump	30
7.7	Cara Menjawab Soal Manual	30
7.7.1	Format Jawaban KMP	30

7.7.2	Format Jawaban Boyer-Moore	31
7.8	Kesalahan Umum dan Edge Case	31
7.9	Checklist Pemahaman	31
7.9.1	Submateri yang Telah Dibahas	31
7.9.2	Prioritas Belajar	32
8	String Matching with Regex	32
8.1	Outline Fundamental Konsep Materi	32
8.2	Pentingnya Materi dan Relevansinya	33
8.2.1	Mengapa Materi Ini Penting	33
8.2.2	Implementasi Dasar	33
8.2.3	Relevansi dengan Industri Saat Ini	33
8.2.4	Dependensi dan Hubungan dengan Materi Lain	33
8.3	Pola Soal UAS Sebelumnya dan Prioritas	33
8.3.1	Formula Eksplisit yang Perlu dikuasai	34
8.4	Penjelasan Konsep Fundamental	35
8.4.1	Definisi Regex	35
8.4.2	Character Class	35
8.4.3	Quantifier dan Boundary Matcher	35
8.5	Komponen Kunci Penting	35
8.6	Algoritma dan Rumus Penting	35
8.6.1	Penggunaan Dasar Regex di Python	35
8.7	Checklist Pemahaman	36
8.7.1	Submateri yang Telah Dibahas	36
8.7.2	Prioritas Belajar	36
9	Program Dinamis	36
9.1	Outline Fundamental Konsep Materi	36
9.2	Pentingnya Materi dan Relevansinya	37
9.2.1	Dependensi dan Hubungan dengan Materi Lain	38
9.3	Pola Soal UAS Sebelumnya dan Prioritas	38
9.4	Penjelasan Konsep Fundamental	38
9.4.1	Definisi Program Dinamis	38
9.4.2	Tahap, Status, Keputusan, dan Transisi	39
9.4.3	Prinsip Optimalitas	39
9.4.4	Memoization dan Tabulation	39
9.4.5	Program Dinamis Maju dan Mundur	40
9.5	Komponen Kunci Penting	40
9.6	Algoritma dan Rumus Penting	40
9.6.1	Skema Umum Program Dinamis	40
9.6.2	Shortest Path Bertahap	41
9.6.3	House Robber atau Coin-Row	41
9.6.4	Integer Knapsack	42
9.6.5	Coin Exchange	42
9.6.6	TSP dengan Program Dinamis	42
9.7	Cara Menjawab Soal Manual	43
9.7.1	Template Jawaban Formulasi DP	43
9.7.2	Template Tabel Shortest Path	43
9.8	Kesalahan Umum dan Edge Case	43
9.9	Checklist Pemahaman	44
9.9.1	Submateri yang Telah Dibahas	44
9.9.2	Prioritas Belajar	44

10 Teori P, NP, NPC	44
10.1 Outline Fundamental Konsep Materi	45
10.2 Pentingnya Materi dan Relevansinya	46
10.2.1 Dependensi dan Hubungan dengan Materi Lain	46
10.3 Pola Soal UAS Sebelumnya dan Prioritas	46
10.4 Penjelasan Konsep Fundamental	47
10.4.1 Persoalan Keputusan dan Persoalan Optimisasi	47
10.4.2 Polynomial-time, Tractable, dan Intractable	47
10.4.3 Decidable dan Undecidable	47
10.4.4 Algoritma Deterministik dan Non-deterministik	47
10.4.5 Kelas P	48
10.4.6 Kelas NP	48
10.4.7 NP-Complete	48
10.4.8 NP-Hard	48
10.5 Komponen Kunci Penting	49
10.6 Algoritma dan Rumus Penting	49
10.6.1 Kelas P	49
10.6.2 Kelas NP	49
10.6.3 Relasi P dan NP	50
10.6.4 Reduksi Polinomial	50
10.6.5 Syarat NP-Complete dan NP-Hard	50
10.7 Contoh Klasifikasi yang Sering Muncul	51
10.8 Jebakan Benar/Salah yang Perlu Diwaspadai	51
10.9 Cara Menjawab Soal Manual	51
10.9.1 Menentukan Apakah Persoalan Berada di NP	51
10.9.2 Membuktikan NP-Complete	52
10.9.3 Menganalisis Konsekuensi Algoritma Polinomial	52
10.10 Checklist Pemahaman	52
10.10.1 Submateri yang Telah Dibahas	52
10.10.2 Prioritas Belajar	52

1 Informasi Ujian

Jenis Ujian	Ujian Akhir Semester
Nama Mata Kuliah	IF2221 Strategi Algoritma
Tanggal Ujian	Rabu, 3 Juni 2026

Tabel 1: Informasi ujian

1.1 Capaian Pembelajaran Mata Kuliah (CPMK)

1. **CPMK1.** Menyusun solusi masalah komputasi dengan berbagai strategi algoritma, bergantung pada karakteristik persoalan, kemudian mendesain algoritma penyelesaian masalah
2. **CPMK2.** Mengimplementasikan desain algoritma menjadi sebuah program aplikasi
3. **CPMK3.** Menulis makalah ilmiah yang berisi hasil studi dan implementasi penyelesaian masalah dengan menggunakan berbagai strategi algoritma.
4. **CPMK4.** Menulis laporan penyelesaian masalah komputasi yang diberikan di dalam tugas
5. **CPMK5.** Mendemokan aplikasi untuk mendapatkan feedback

1.2 Tabel Hubungan CPMK dan Materi

CPMK	BT	B&B	RP	SM	Regex	PD	PNP
CPMK1	✓	✓	✓	✓	✓	✓	✓
CPMK2	✓	✓	✓	✓	✓	✓	
CPMK3	✓	✓	✓	✓	✓	✓	✓
CPMK4	✓	✓	✓	✓	✓	✓	✓
CPMK5	✓	✓	✓	✓	✓	✓	

Tabel 2: Hubungan CPMK dengan materi yang dibahas

Keterangan: BT = Backtracking, B&B = Branch and Bound, RP = Route Planning, SM = String Matching, PD = Program Dinamis, PNP = Teori P, NP, NPC.

2 Daftar Materi yang Diujikan

1. Backtracking
2. Branch and Bound
3. Route Planning
4. String Matching
5. String Matching with Regex
6. Program Dinamis
7. Teori P, NP, NPC

3 Outline Keseluruhan Materi

Bagian ini menyajikan peta belajar ringkas untuk seluruh materi yang diujikan. Penjelasan mendalam untuk setiap materi tersedia pada bagian catatan materi masing-masing.

3.1 Backtracking

- Runut-balik sebagai perbaikan dari *exhaustive search* dengan pemangkasan simpul yang tidak mengarah ke solusi.
- Pohon ruang status, simpul hidup, simpul-E, simpul mati, fungsi pembangkit, dan fungsi pembatas.
- Aplikasi: N-Ratu, *sum of subsets*, pewarnaan graf, sirkuit Hamilton, dan rute labirin.

3.2 Branch and Bound

- Strategi optimisasi berbasis pohon ruang status, *best-first rule*, dan nilai batas.
- Fungsi ongkos untuk memilih simpul hidup yang paling menjanjikan.
- Aplikasi: 15-Puzzle, TSP, *assignment problem*, dan 1/0 Knapsack.

3.3 Route Planning

- Pencarian rute pada graf dari status awal menuju status tujuan.
- *Uninformed search*: BFS, DFS, DLS, IDS, UCS.
- *Informed search*: Greedy Best-First Search dan A* dengan fungsi evaluasi.

3.4 String Matching

- Pencarian lokasi kemunculan *pattern* di dalam teks.
- Algoritma Brute Force, Knuth-Morris-Pratt (KMP), dan Boyer-Moore.
- Fungsi pinggiran KMP dan *last occurrence function* Boyer-Moore.

3.5 String Matching with Regex

- Regular expression sebagai notasi pola string yang lebih fleksibel daripada *exact matching*.
- Character class, range, negasi, metacharacter, quantifier, dan boundary matcher.
- Penggunaan dasar regex di Python melalui `re.compile`, `search`, `match`, dan `findall`.

3.6 Program Dinamis

- Metode optimisasi berbasis tahap, status, keputusan, prinsip optimalitas, dan memoisasi.
- Pendekatan program dinamis maju dan mundur.
- Aplikasi: lintasan terpendek, integer knapsack, coin exchange, capital budgeting, dan TSP.

3.7 Teori P, NP, NPC

- Klasifikasi persoalan berdasarkan keterpecahan dan kompleksitas waktu.
- Polynomial-time, nonpolynomial-time, tractable, intractable, undecidable, deterministik, dan non-deterministik.
- Kelas P, NP, NP-Complete, NP-Hard, reduksi polinomial, SAT, dan TSP decision problem.

4 Backtracking

Backtracking atau runut-balik adalah metode pemecahan masalah yang membangun solusi secara bertahap di dalam pohon ruang status. Fokus utama untuk UAS adalah memahami bagaimana kandidat solusi dibangkitkan, diuji dengan fungsi pembatas, lalu dipangkas ketika tidak lagi mungkin mengarah ke solusi.

Materi ini berada setelah DFS/BFS karena pola penelusurannya mengikuti *depth-first order*. Backtracking menjadi fondasi penting sebelum Branch and Bound, karena keduanya sama-sama membentuk pohon ruang status dan melakukan pemangkasan, tetapi berbeda dalam aturan ekspansi dan orientasi optimisasinya.

4.1 Outline Fundamental Konsep Materi

1. Definisi dan posisi backtracking
 - (a) Perbaikan dari *exhaustive search*
 - (b) Hubungan dengan DFS
 - (c) Optimasi dan non-optimasi
2. Properti umum
 - (a) Solusi sebagai vektor $X = (x_1, x_2, \dots, x_n)$
 - (b) Himpunan nilai S_i
 - (c) Fungsi pembangkit T
 - (d) Fungsi pembatas B
3. Pohon ruang status
 - (a) Ruang solusi dan pohon berakar
 - (b) Simpul hidup, simpul-E, dan simpul mati
 - (c) *Pruning*
4. Skema algoritma
 - (a) Versi rekursif
 - (b) Versi iteratif
 - (c) Kompleksitas kasus terburuk
5. Aplikasi
 - (a) N-Ratu
 - (b) Sum of Subsets
 - (c) Pewarnaan graf
 - (d) Sirkuit Hamilton
 - (e) Labirin

4.2 Pentingnya Materi dan Relevansinya

4.2.1 Mengapa Materi Ini Penting

Backtracking penting karena banyak persoalan strategi algoritma memiliki ruang solusi besar, tetapi tidak semua kemungkinan perlu dievaluasi. Dengan fungsi pembatas, pencarian dapat dihentikan lebih awal pada cabang yang melanggar kendala. Kemampuan utama yang dibangun adalah memodelkan solusi sebagai keputusan bertahap, menentukan kendala, dan menulis aturan pemangkasan yang benar.

4.2.2 Implementasi Dasar

Implementasi dasar backtracking biasanya berbentuk prosedur rekursif. Tiap pemanggilan menentukan satu komponen solusi, memeriksa apakah solusi parsial masih menjanjikan, lalu melanjutkan ke komponen berikutnya jika memenuhi kendala.

4.2.3 Relevansi dengan Industri Saat Ini

Komentar 4.1: Keterbatasan Sumber

Sumber utama tidak memberikan pembahasan industri eksplisit untuk backtracking. Bagian ini dibatasi pada aplikasi yang disebut di sumber, yaitu pemilihan subset, penjadwalan/pemilihan kombinasi, pewarnaan graf, sirkuit Hamilton, dan pencarian rute labirin.

4.2.4 Dependensi dan Hubungan dengan Materi Lain

Backtracking bergantung pada pemahaman DFS dan pohon. Materi ini berhubungan langsung dengan Branch and Bound karena keduanya memakai pohon ruang status dan pemangkasan, tetapi Branch and Bound menggunakan nilai batas untuk persoalan optimisasi.

4.3 Pola Soal UAS Sebelumnya dan Prioritas

Komentar 4.2: Pola dari Soal 2022–2025

Backtracking muncul pada semua sumber soal yang diperiksa: 2022, solusi 2023, 2024, dan 2025. Bentuk soal berulang adalah merumuskan representasi solusi, domain variabel, fungsi pembangkit, fungsi pembatas, dan proses pembentukan pohon ruang status.

Bagian yang paling perlu diprioritaskan adalah pemodelan persoalan sebagai vektor solusi, domain variabel, fungsi pembangkit, dan fungsi pembatas. Pada 2022, bentuk soal mencakup benar/salah untuk pewarnaan graf dan constraint cryptarithmic, serta penentuan variabel dan domain Magic Square. Pada solusi 2023, soal meminta representasi solusi, domain, batasan, dan satu solusi untuk penjadwalan. Pada 2024, cryptarithmic kembali muncul dengan permintaan representasi solusi, fungsi pembangkit, dan constraints. Pada 2025, soal meminta pohon ruang status untuk pewarnaan graf dan deskripsi fungsi pembatas.

- **Pola desain:** rumuskan X , domain tiap variabel, fungsi pembangkit T , dan fungsi pembatas B untuk cryptarithmic, penjadwalan, magic square, atau pewarnaan graf.
- **Pola penelusuran:** gambarkan pohon ruang status dan tandai cabang yang mati karena melanggar constraint.
- **Pola benar/salah:** evaluasi apakah constraint tertentu sudah cukup benar untuk fungsi pembatas.

Permasalahan yang Sering Muncul	Yang Biasanya Diminta	Fungsi/Komponen Wajib Dipahami
Cryptarithmic	Representasi solusi, domain digit, constraint huruf berbeda, dan constraint aritmetika kolom.	Vektor solusi X , domain S_i , fungsi pembangkit T , fungsi pembatas B .
Pewarnaan graf	Pohon ruang status pewarnaan simpul dan cabang yang dipangkas karena simpul bertetangga berwarna sama.	Fungsi pembatas adjacency, urutan pewarnaan simpul, simpul hidup, simpul-E, simpul mati.
Penjadwalan dan magic square	Variabel keputusan, domain waktu/angka, constraint antar-variabel, dan solusi feasible.	Model solusi parsial, constraint feasibility, pruning.

Tabel 3: Permasalahan dan fungsi penting pada Backtracking

4.3.1 Formula Eksplisit yang Perlu dikuasai

Inti Formula.

Untuk setiap persoalan backtracking, tuliskan solusi sebagai $X = (x_1, x_2, \dots, x_n)$, tentukan domain $x_i \in S_i$, lalu definisikan fungsi pembatas $B(x_1, \dots, x_k)$.

Pada pewarnaan graf, jika x_i menyatakan warna simpul v_i , maka fungsi pembatasnya:

$$B(x_1, \dots, x_k) = \text{true} \iff \forall (v_i, v_j) \in E, i, j \leq k \Rightarrow x_i \neq x_j \quad (1)$$

Pada cryptarithmic, jika huruf a dipetakan ke digit $d(a)$, maka constraint dasarnya:

$$d(a) \in \{0, 1, \dots, 9\}, \quad (2)$$

$$a \neq b \Rightarrow d(a) \neq d(b), \quad (3)$$

$$d(a_{\text{awal}}) \neq 0. \quad (4)$$

Constraint aritmetika dibuat per kolom. Misalnya pada kolom satuan:

$$d(A) + d(B) + c_{\text{in}} = d(C) + 10c_{\text{out}} \quad (5)$$

dengan c_{in} dan c_{out} adalah carry masuk dan keluar.

Pada magic square $n \times n$, target jumlah setiap baris, kolom, dan diagonal adalah:

$$M = \frac{n(n^2 + 1)}{2} \quad (6)$$

Fungsi pembatas mengecek agar jumlah parsial tidak melebihi M dan baris/kolom/diagonal lengkap harus sama dengan M .

4.4 Penjelasan Konsep Fundamental

4.4.1 Backtracking sebagai Perbaikan Exhaustive Search

Inti Konsep.

Backtracking mengeksplorasi hanya pilihan yang masih mengarah ke solusi dan memangkas pilihan yang melanggar kendala.

Pada *exhaustive search*, semua kemungkinan solusi dieksplorasi dan dievaluasi satu per satu. Pada backtracking, pencarian tetap sistematis, tetapi solusi parsial yang sudah terbukti tidak mungkin menjadi solusi lengkap tidak diteruskan. Karena itu, backtracking tidak mengubah ruang solusi, tetapi mengurangi bagian ruang solusi yang benar-benar dikunjungi.

4.4.2 Properti Umum Algoritma Runut-balik

Inti Konsep.

Setiap persoalan backtracking perlu dirumuskan sebagai vektor solusi, fungsi pembangkit nilai, dan fungsi pembatas.

Solusi dinyatakan sebagai vektor $X = (x_1, x_2, \dots, x_n)$ dengan $x_i \in S_i$. Fungsi pembangkit $T(x_1, x_2, \dots, x_{k-1})$ menghasilkan kandidat nilai untuk x_k . Fungsi pembatas $B(x_1, x_2, \dots, x_k)$ bernilai benar jika solusi parsial masih mengarah ke solusi, dan salah jika solusi parsial harus dibuang.

Mekanisme atau Prosedur Kerja

1. Nyatakan solusi sebagai vektor keputusan.
2. Tentukan domain nilai untuk setiap komponen solusi.
3. Bangkitkan nilai komponen berikutnya dengan fungsi T .
4. Uji solusi parsial menggunakan fungsi B .
5. Lanjutkan jika B benar; lakukan backtrack jika B salah.

4.4.3 Pohon Ruang Status

Inti Konsep.

Pohon ruang status mengorganisasikan seluruh kemungkinan solusi sebagai lintasan dari akar menuju daun.

Setiap simpul menyatakan status persoalan. Setiap sisi menyatakan pilihan nilai untuk salah satu komponen solusi. Simpul yang sudah dibangkitkan disebut simpul hidup, simpul hidup yang sedang diperluas disebut simpul-E, dan simpul yang tidak lagi mengarah ke solusi disebut simpul mati.

Klasifikasi atau Perbandingan

- **Simpul hidup:** simpul yang sudah dibangkitkan dan masih mungkin diperluas.
- **Simpul-E:** simpul hidup yang sedang diperluas.
- **Simpul mati:** simpul yang dipangkas karena tidak mengarah ke solusi.

4.4.4 Aplikasi Backtracking

Inti Konsep.

Aplikasi backtracking berbeda pada domain nilai dan fungsi pembatas, tetapi sama-sama membangun solusi parsial secara bertahap.

Pada N-Ratu, x_i menyatakan kolom ratu pada baris ke- i , dan fungsi pembatas memastikan tidak ada dua ratu pada kolom atau diagonal yang sama. Pada *sum of subsets*, $x_i \in \{0, 1\}$ menyatakan apakah bobot w_i dipilih, dan fungsi pembatas memastikan jumlah bobot masih mungkin mencapai target. Pada pewarnaan graf, x_i menyatakan warna simpul ke- i , dan fungsi pembatas memastikan simpul bertetangga tidak memiliki warna sama.

4.5 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
$X = (x_1, x_2, \dots, x_n)$	Vektor solusi yang dibangun bertahap.
S_i	Himpunan nilai yang mungkin untuk komponen x_i .
T	Fungsi pembangkit kandidat nilai komponen solusi berikutnya.
B	Fungsi pembatas yang menentukan apakah solusi parsial masih menjanjikan.
Pohon ruang status	Struktur pohon yang merepresentasikan ruang solusi.
<i>Pruning</i>	Pemangkasan simpul yang tidak mungkin menghasilkan solusi.

Tabel 4: Komponen kunci dalam materi Backtracking

4.6 Algoritma dan Rumus Penting

4.6.1 Skema Umum Backtracking Rekursif

Tujuan Algoritma.

Skema ini mencari semua solusi dengan membangun komponen solusi satu per satu dan memangkas solusi parsial yang tidak memenuhi fungsi pembatas.

Listing 1: Pseudocode backtracking rekursif

```

1 procedure RunutBalikR(k)
2   for setiap x[k] dalam T(x[1], ..., x[k-1]) do
3     if B(x[1], ..., x[k]) = true then
4       if (x[1], ..., x[k]) adalah solusi then
5         cetak(x[1], ..., x[k])
6       endif
7       if k < n then
8         RunutBalikR(k + 1)
9       endif
10    endif
11  endfor
12
13 Pemanggilan awal: RunutBalikR(1)

```

Dalam kasus terburuk, jika jumlah simpul pohon ruang status adalah 2^n atau $n!$, maka waktu backtracking dapat ditulis sebagai $O(p(n)2^n)$ atau $O(q(n)n!)$, dengan $p(n)$ dan $q(n)$ menyatakan waktu komputasi pada setiap simpul.

4.6.2 Fungsi Pembatas Sum of Subsets

Makna Rumus.

Rumus ini menentukan apakah solusi parsial pada persoalan *sum of subsets* masih mungkin mencapai target m .

$$\begin{aligned}
 B(x_1, \dots, x_k) = \text{true} \iff & \sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i \geq m \\
 & \wedge \left(\sum_{i=1}^k w_i x_i = m \vee \sum_{i=1}^k w_i x_i + w_{k+1} \leq m \right)
 \end{aligned} \tag{7}$$

dengan:

- w_i : bobot elemen ke- i .
- x_i : bernilai 1 jika w_i dipilih, dan 0 jika tidak dipilih.
- m : target jumlah bobot.

4.7 Checklist Pemahaman

4.7.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan backtracking sebagai perbaikan dari *exhaustive search*.
- ☐ Saya dapat membedakan ruang solusi, pohon ruang status, simpul hidup, simpul-E, dan simpul mati.
- ☐ Saya dapat merumuskan fungsi pembangkit dan fungsi pembatas untuk persoalan sederhana.
- ☐ Saya dapat menjelaskan skema rekursif backtracking.
- ☐ Saya dapat menelusuri cabang yang dipangkas pada contoh backtracking sederhana.

4.7.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Vektor solusi, pohon ruang status, fungsi pembangkit, fungsi pembatas, pruning, dan skema rekursif.
Cukup Pahami Konsep	Contoh aplikasi N-Ratu, Sum of Subsets, pewarnaan graf, sirkuit Hamilton, dan labirin.
Sudah dikuasai	Belum ditentukan.

5 Branch and Bound

Branch and Bound (B&B) adalah strategi untuk persoalan optimisasi yang membentuk pohon ruang status dan memilih simpul berikutnya berdasarkan nilai batas atau *cost*. Fokus utama UAS adalah memahami perbedaan B&B dari backtracking, cara kerja *least cost search*, dan penggunaan nilai batas untuk memangkas simpul.

5.1 Outline Fundamental Konsep Materi

- Definisi B&B
 - Persoalan optimisasi
 - Pohon ruang status
 - BFS dan *least cost search*
- Perbandingan dengan backtracking
 - Persamaan: pohon ruang status dan memangkas
 - Perbedaan: DFS vs *best-first rule*
- Fungsi batas
 - Lower bound untuk minimisasi
 - Upper bound untuk maksimasi
 - Solusi terbaik sejauh ini

4. Aplikasi

- (a) 15-Puzzle
- (b) Travelling Salesperson Problem
- (c) Assignment Problem
- (d) 1/0 Knapsack

5.2 Pentingnya Materi dan Relevansinya

5.2.1 Mengapa Materi Ini Penting

B&B penting karena memberi cara sistematis untuk menangani persoalan optimisasi yang ruang solusinya sangat besar. Nilai batas membantu menentukan cabang yang masih layak dieksplorasi dan cabang yang tidak dapat mengalahkan solusi terbaik sejauh ini.

5.2.2 Implementasi Dasar

Implementasi dasar B&B menyimpan simpul hidup di dalam struktur prioritas. Untuk minimisasi, simpul dengan cost terkecil diekspansi lebih dahulu. Untuk maksimasi, simpul dengan batas atas terbesar menjadi kandidat ekspansi.

5.2.3 Relevansi dengan Industri Saat Ini

Komentar 5.1: Keterbatasan Sumber

Sumber utama membahas relevansi melalui contoh persoalan optimisasi seperti TSP, assignment problem, 15-Puzzle, dan knapsack. Catatan ini tidak menambahkan klaim industri di luar contoh yang tersedia pada sumber.

5.2.4 Dependensi dan Hubungan dengan Materi Lain

B&B bergantung pada pemahaman pohon ruang status, BFS, dan backtracking. Materi ini juga terkait dengan Route Planning dan Program Dinamis karena sama-sama menangani pencarian solusi dengan biaya atau nilai optimal.

5.3 Pola Soal UAS Sebelumnya dan Prioritas

Komentar 5.2: Pola dari Soal 2022–2025

Branch and Bound muncul pada semua sumber soal yang diperiksa. Pola paling berulang adalah membangun pohon ruang status, menghitung bound/cost simpul, memilih simpul-E, dan mematikan simpul yang tidak menjanjikan.

Prioritas utama adalah assignment problem, knapsack, TSP, dan n-puzzle. Pada 2022, soal meminta pohon ruang status n-puzzle dengan Manhattan distance, perhitungan TSP memakai *reduced cost matrix* dan bobot tur lengkap, serta pohon B&B knapsack sampai level 1. Pada solusi 2023, soal assignment minimisasi meminta pohon, bound setiap simpul, simpul yang dibunuh, sampai goal node. Pada 2024, bentuknya knapsack maksimasi. Pada 2025, bentuknya assignment maksimasi profit.

- **Pola hitung manual:** hitung $\hat{c}(i)$, bound assignment, bound knapsack, atau reduksi matriks TSP.
- **Pola penelusuran:** tentukan simpul hidup, simpul-E, solusi terbaik sejauh ini, dan simpul yang dimatikan.

- **Pola aplikasi:** gunakan B&B untuk minimisasi waktu assignment, maksimasi profit assignment, maksimasi knapsack, TSP, atau n-puzzle.

Permasalahan yang Sering Muncul	Yang Biasanya Diminta	Fungsi/Komponen Wajib Dipahami
Assignment problem	Menentukan assignment minimum/maksimum, membangun pohon ruang status, menghitung bound, dan memilih goal node.	Fungsi cost $\hat{c}(i)$, bound assignment, solusi terbaik sejauh ini, priority queue.
0/1 Knapsack	Membentuk pohon keputusan ambil/tidak ambil dan menghitung bound untuk memaksimalkan profit.	Upper bound profit, kapasitas tersisa, feasibility, pruning berdasarkan best profit.
TSP dan n-puzzle	Menghitung cost simpul TSP dengan <i>reduced cost matrix</i> atau menghitung heuristik n-puzzle.	$\hat{c}(i) = \hat{f}(i) + \hat{g}(i)$, reduced cost, Manhattan distance, simpul-E.

Tabel 5: Permasalahan dan fungsi penting pada Branch and Bound

5.3.1 Formula Eksplisit yang Perlu dikuasai

Inti Formula.

Pada Branch and Bound, setiap simpul harus memiliki bound. Untuk minimisasi, simpul dengan bound terkecil diekspansi dan simpul dengan bound $\geq$ solusi terbaik dipangkas. Untuk maksimasi, arah perbandingannya dibalik.

Fungsi cost umum:

$$\hat{c}(i) = \hat{f}(i) + \hat{g}(i) \quad (8)$$

dengan $\hat{f}(i)$ sebagai biaya dari akar ke simpul i , dan $\hat{g}(i)$ sebagai taksiran biaya terbaik dari simpul i ke solusi lengkap.

Pada assignment problem minimisasi, jika simpul sudah menetapkan sebagian pasangan pekerja-pekerjaan, bound dapat dihitung:

$$LB(u) = \text{cost_so_far}(u) + \sum_{r \in R(u)} \min_{j \in J(u)} C_{rj} \quad (9)$$

dengan $R(u)$ adalah pekerja yang belum diberi pekerjaan, $J(u)$ pekerjaan yang belum dipakai, dan C_{rj} biaya pekerja r mengerjakan pekerjaan j .

Pada 0/1 knapsack maksimasi, bound yang umum dipakai adalah relaksasi fractional knapsack:

$$UB(u) = p(u) + \sum_{\text{barang penuh}} p_i + \frac{K - w(u) - \sum_{\text{barang penuh}} w_i}{w_t} p_t \quad (10)$$

dengan K kapasitas, $p(u)$ profit sementara, $w(u)$ bobot sementara, dan barang diurutkan berdasarkan rasio p_i/w_i .

Pada TSP dengan reduced cost matrix:

$$\hat{c}(S) = \hat{c}(R) + A(i, j) + r \quad (11)$$

dengan R simpul orang tua, S simpul anak, $A(i, j)$ biaya sisi pada matriks tereduksi, dan r total reduksi baris-kolom baru.

Pada n-puzzle, cost simpul biasanya:

$$\hat{c}(i) = g(i) + h(i) \quad (12)$$

dengan $g(i)$ jumlah langkah dari akar dan $h(i)$ jumlah Manhattan distance semua tile terhadap posisi goal.

5.4 Penjelasan Konsep Fundamental

5.4.1 Definisi Branch and Bound

Inti Konsep.

B&B adalah BFS yang dimodifikasi dengan *least cost search*, sehingga simpul hidup yang diekspansi adalah simpul dengan nilai batas terbaik.

Setiap simpul diberi nilai $\hat{c}(i)$, yaitu taksiran lintasan termurah ke simpul tujuan yang melalui simpul status i . Pada kasus minimisasi, simpul dengan $\hat{c}(i)$ terkecil dipilih sebagai simpul-E.

5.4.2 Pemangkasan pada B&B

Inti Konsep.

Simpul dipangkas jika nilainya tidak lebih baik dari solusi terbaik sejauh ini, tidak feasible, atau sudah tidak memiliki pilihan lanjutan yang berguna.

Pemangkasan tidak hanya bergantung pada pelanggaran kendala, tetapi juga pada nilai objektif. Jika batas bawah suatu simpul minimisasi lebih buruk daripada solusi terbaik sejauh ini, simpul itu tidak perlu diekspansi.

5.4.3 Aplikasi B&B

Inti Konsep.

Contoh utama B&B pada sumber adalah 15-Puzzle, TSP, Assignment Problem, dan 1/0 Knapsack.

Pada TSP, nilai batas dapat dihitung dengan matriks ongkos tereduksi atau bobot tur lengkap. Pada assignment problem, lower bound dapat dihitung dari nilai minimum per baris/kolom. Pada 1/0 knapsack, karena persoalan maksimasi, nilai batas simpul menyatakan batas atas dari nilai optimum.

5.5 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
$\hat{c}(i)$	Nilai taksiran cost atau bound untuk simpul i .
Simpul hidup	Simpul yang sudah dibangkitkan tetapi belum dimatikan.
Simpul-E	Simpul hidup yang dipilih untuk diekspansi.
<i>Least cost search</i>	Aturan memilih simpul dengan cost terkecil pada persoalan minimisasi.
Solusi terbaik sejauh ini	Acuan untuk mematikan simpul yang tidak dapat memberi solusi lebih baik.

Tabel 6: Komponen kunci dalam materi Branch and Bound

5.6 Algoritma dan Rumus Penting

5.6.1 Cost Simpul

Makna Rumus.

Cost simpul memperkirakan ongkos solusi terbaik yang dapat dicapai melalui simpul tersebut.

$$\hat{c}(i) = \hat{f}(i) + \hat{g}(i) \quad (13)$$

dengan:

- $\hat{c}(i)$: cost simpul i .
- $\hat{f}(i)$: ongkos mencapai simpul i dari akar.
- $\hat{g}(i)$: taksiran ongkos dari simpul i ke tujuan.

5.6.2 Bound TSP dengan Reduced Cost Matrix

Makna Rumus.

Rumus ini menghitung batas bawah anak simpul pada TSP setelah memilih sisi (i, j) dan melakukan reduksi ulang matriks.

$$\hat{c}(S) = \hat{c}(R) + A(i, j) + r \quad (14)$$

dengan:

- R : simpul orang tua.
- S : simpul anak.
- $A(i, j)$: elemen matriks tereduksi yang bersesuaian dengan sisi (i, j) .
- r : total pengurang dari reduksi baris dan kolom berikutnya.

5.7 Checklist Pemahaman

5.7.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan B&B sebagai BFS dengan *least cost search*.
- ☐ Saya dapat membedakan B&B dan backtracking.
- ☐ Saya dapat menjelaskan kriteria pemangkasan simpul.
- ☐ Saya dapat menjelaskan makna lower bound dan upper bound.
- ☐ Saya dapat menghitung bound sederhana dan menentukan simpul yang dipangkas.

5.7.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Konsep bound, pemilihan simpul-E, pruning, perbedaan dengan backtracking, dan rumus TSP reduced cost.
Cukup Pahami Konsep	15-Puzzle, Assignment Problem, dan 1/0 Knapsack.
Sudah dikuasai	Belum ditentukan.

6 Route Planning

Route Planning adalah persoalan mencari lintasan dari simpul awal menuju simpul tujuan pada graf. Dalam konteks Strategi Algoritma, materi ini bukan sekadar “mencari jalan”, tetapi memahami bagaimana algoritma pencarian memilih simpul berikutnya, bagaimana biaya lintasan dihitung, bagaimana heuristik memengaruhi keputusan, dan kapan hasil pencarian dapat dipercaya sebagai lintasan optimal.

Fokus utama UAS adalah menjalankan algoritma secara manual pada graf berbobot: menentukan simpul ekspansi, memperbarui simpul hidup, menghitung nilai evaluasi, menilai admissibility heuristik, lalu menuliskan jalur, jarak total, dan banyak iterasi. Pola ini muncul konsisten pada soal UAS 2022, solusi UAS 2023, soal UAS 2024, dan soal UAS 2025 yang tersedia di folder proyek.

Komentar 6.1: Keterbatasan Sumber Utama

NotebookLM yang diberikan dapat diakses, tetapi jawaban yang diperoleh untuk Route Planning hanya memuat outline global: route planning sebagai pencarian lintasan terbaik, kategori *uninformed search* dan *informed search*, serta rumus $f(n) = g(n)$ untuk UCS, $f(n) = h(n)$ untuk Greedy Best-First Search, dan $f(n) = g(n) + h(n)$ untuk A*. Detail tambahan seperti syarat optimalitas, konsistensi heuristik, kompleksitas, dan prosedur implementasi dilengkapi menggunakan pengetahuan umum yang relevan untuk memperdalam pemahaman.

6.1 Outline Fundamental Konsep Materi

1. Model persoalan route planning
 - (a) Graf sebagai representasi lokasi, status, atau konfigurasi.
 - (b) Simpul awal, simpul tujuan, sisi, bobot sisi, dan biaya lintasan.
 - (c) *Goal test*: kondisi yang menentukan bahwa pencarian sudah mencapai tujuan.
 - (d) Perbedaan graf berarah dan graf tidak berarah.
 - (e) Perbedaan graf berbobot dan tidak berbobot.
2. Terminologi pencarian
 - (a) Simpul hidup (*live node/frontier/agenda*).
 - (b) Simpul ekspansi atau simpul-E (*expanded node*).
 - (c) Simpul yang sudah dikunjungi atau ditutup (*explored/closed set*).
 - (d) Jalur parsial, parent pointer, dan rekonstruksi jalur.
 - (e) Aturan *tie-breaking*, misalnya urutan abjad.
3. Fungsi biaya dan fungsi evaluasi
 - (a) $g(n)$: biaya aktual dari simpul awal ke simpul n .
 - (b) $h(n)$: estimasi biaya dari simpul n ke simpul tujuan.
 - (c) $h^*(n)$: biaya optimal sebenarnya dari simpul n ke tujuan.
 - (d) $f(n)$: nilai prioritas yang digunakan untuk memilih simpul berikutnya.
4. *Uninformed search* atau *blind search*
 - (a) Breadth-First Search (BFS).
 - (b) Depth-First Search (DFS).
 - (c) Depth-Limited Search (DLS).

- (d) Iterative Deepening Search (IDS).
 - (e) Uniform Cost Search (UCS).
5. *Informed search* atau *heuristic search*
- (a) Greedy Best-First Search.
 - (b) A*.
 - (c) Heuristik admissible.
 - (d) Heuristik consistent atau monotone.
 - (e) Hubungan kualitas heuristik dengan jumlah iterasi.
6. Pola pengerjaan manual untuk UAS
- (a) Membuat tabel iterasi.
 - (b) Menghitung $g(n)$, $h(n)$, dan $f(n)$ untuk simpul hidup.
 - (c) Menghapus atau mengabaikan simpul yang sudah menjadi simpul ekspansi sesuai aturan soal.
 - (d) Menghentikan pencarian saat simpul tujuan menjadi simpul ekspansi, bukan saat pertama kali muncul di simpul hidup.
 - (e) Menuliskan jalur hasil, jarak total, banyak iterasi, dan analisis singkat.

6.2 Pentingnya Materi dan Relevansinya

Route Planning penting karena ia memperlihatkan bentuk paling konkret dari persoalan pencarian pada graf. Banyak persoalan komputasi dapat dimodelkan sebagai pencarian lintasan: simpul menyatakan keadaan, sisi menyatakan aksi yang mengubah keadaan, dan bobot sisi menyatakan biaya aksi. Dengan model ini, persoalan navigasi kota, pencarian solusi puzzle, perencanaan gerak robot, routing paket jaringan, bahkan pencarian urutan keputusan dalam sistem otomatis dapat dipelajari memakai kerangka yang sama.

Dalam ujian, nilai utama materi ini biasanya bukan pada hafalan nama algoritma, melainkan pada kemampuan menjaga invariant tabel pencarian. Jika pada setiap iterasi mahasiswa konsisten memilih simpul dengan nilai evaluasi terkecil, menghitung ulang biaya lintasan dengan benar, dan berhenti pada kondisi yang tepat, jawaban akan tetap dapat dipertanggungjawabkan meskipun graf soal cukup kompleks.

Komentar 6.2: Relevansi Praktis

Dalam praktik industri, route planning muncul pada sistem navigasi, logistik, robotika, game AI, pemetaan rute kendaraan, jaringan komputer, dan perencanaan aksi. Catatan ini membatasi pembahasan pada bentuk algoritmik yang relevan untuk UAS: graf, biaya, heuristik, frontier, dan tabel iterasi.

6.2.1 Dependensi dan Hubungan dengan Materi Lain

Route Planning berhubungan langsung dengan Backtracking karena sama-sama dapat dipandang sebagai eksplorasi ruang status. Perbedaannya, Backtracking biasanya menekankan pembentukan solusi bertahap dan pruning berdasarkan constraint, sedangkan Route Planning menekankan urutan ekspansi simpul berdasarkan struktur frontier dan fungsi evaluasi.

Materi ini juga dekat dengan Branch and Bound karena UCS dan A* sama-sama memilih simpul hidup berdasarkan nilai prioritas. Pada Branch and Bound, nilai prioritas adalah bound atau cost optimistik dari simpul ruang status; pada Route Planning, nilai prioritas dinyatakan eksplisit sebagai $g(n)$, $h(n)$, atau $g(n) + h(n)$. Hubungannya dengan Program Dinamis muncul pada persoalan lintasan terpendek: keduanya mencari lintasan biaya minimum, tetapi Program Dinamis menyusun relasi tahap-status, sedangkan Route Planning mengekspansi simpul dari frontier.

6.3 Pola Soal UAS Sebelumnya dan Prioritas

Komentar 6.3: Pola dari Soal 2022–2025

Pola soal Route Planning pada dokumen yang tersedia sangat stabil. Soal 2022 meminta uji admissibility dan menjalankan A*, Greedy Best-First Search, serta UCS sampai simpul tujuan menjadi simpul ekspansi. Solusi 2023 menampilkan tabel $g(n)$, $h(n)$, rumus $f(n)$, jalur, cost, dan analisis iterasi. Soal 2024 meminta heuristik semua simpul, admissibility, simpul ekspansi dan simpul hidup pada iterasi tertentu, iterasi berhenti, jalur, dan jarak. Soal 2025 secara eksplisit mensyaratkan jawaban dalam tabel iterasi untuk UCS, Greedy Best-First Search, dan A*.

Pola Soal	Yang Biasanya Diminta	Kemampuan yang Harus dikuasai
Graf berbobot dengan heuristik	Menjalankan UCS, Greedy Best-First Search, dan A* sampai tujuan menjadi simpul ekspansi.	Mengelola frontier, memilih simpul dengan $f(n)$ terkecil, menghitung biaya lintasan, dan merekonstruksi jalur.
Uji heuristik	Menentukan $h(n)$ setiap simpul dan apakah nilai tersebut admissible.	Membandingkan $h(n)$ dengan biaya optimal sebenarnya $h^*(n)$ dari simpul tersebut ke tujuan.
Tabel iterasi	Menuliskan simpul ekspansi, simpul hidup, dan nilai evaluasi setiap simpul hidup.	Membedakan simpul yang baru ditemukan, simpul yang masih hidup, dan simpul yang sudah diekspansi.
Perbandingan algoritma	Menentukan jalur, jarak, jumlah iterasi, dan algoritma yang menghasilkan lintasan terbaik atau iterasi paling sedikit.	Memahami bahwa Greedy dapat cepat tetapi tidak selalu optimal, UCS optimal untuk biaya non-negatif, dan A* optimal jika syarat heuristik terpenuhi.
Aturan tie-breaking	Memilih simpul saat beberapa simpul memiliki nilai $f(n)$ sama.	Mengikuti aturan soal, misalnya urutan abjad, agar tabel deterministik.

Tabel 7: Pola soal UAS Route Planning dan kemampuan yang perlu dikuasai

6.4 Penjelasan Konsep Fundamental

6.4.1 Model Graf untuk Route Planning

Inti Konsep.

Route planning memodelkan persoalan sebagai graf: simpul adalah lokasi atau status, sisi adalah kemungkinan perpindahan, dan bobot sisi adalah biaya perpindahan.

Sebuah persoalan route planning dimulai dari simpul awal s dan simpul tujuan t . Jika graf berbobot, setiap sisi (u, v) memiliki biaya $c(u, v)$ yang biasanya menyatakan jarak, waktu, energi, bahan bakar, atau cost abstrak lain. Tujuan pencarian adalah menemukan lintasan dari s ke t . Jika soal meminta jalur terpendek, maka lintasan yang dicari adalah lintasan dengan total bobot minimum.

Pada graf tidak berbobot, semua sisi dapat dianggap memiliki biaya sama, sehingga jumlah sisi atau jumlah langkah menjadi ukuran biaya. Pada graf berbobot, jumlah langkah belum tentu mencerminkan kualitas lintasan. Jalur dengan lebih banyak sisi dapat saja lebih murah daripada jalur pendek yang melewati sisi berbiaya besar. Inilah alasan UCS dan A* menggunakan $g(n)$, bukan hanya kedalaman simpul.

6.4.2 Frontier, Simpul Ekspan, dan Kondisi Berhenti

Inti Konsep.

Frontier berisi kandidat simpul yang sudah ditemukan tetapi belum diekspansi; simpul ekspan adalah simpul yang sedang dikeluarkan dari frontier untuk diperiksa dan dikembangkan.

Kesalahan umum pada soal UAS adalah berhenti saat simpul tujuan pertama kali muncul di simpul hidup. Instruksi soal 2022, 2024, dan 2025 menekankan bahwa pencarian dihentikan ketika simpul tujuan menjadi simpul ekspan. Artinya, simpul tujuan harus benar-benar terpilih sebagai simpul dengan prioritas terbaik pada suatu iterasi, bukan sekadar sudah ditemukan sebagai tetangga.

Prosedur umum setiap iterasi adalah sebagai berikut:

1. Pilih satu simpul dari frontier berdasarkan aturan algoritma.
2. Jadikan simpul tersebut sebagai simpul ekspan.
3. Jika simpul ekspan adalah tujuan, hentikan pencarian.
4. Bangkitkan tetangga dari simpul ekspan.
5. Hitung nilai $g(n)$, $h(n)$, dan $f(n)$ yang relevan untuk tetangga.
6. Masukkan tetangga ke frontier jika memenuhi aturan soal, misalnya belum pernah menjadi simpul ekspan.
7. Jika suatu simpul sudah ada di frontier melalui jalur lain, simpan jalur dengan prioritas lebih baik apabila aturan algoritma dan soal mengizinkan pembaruan.

Komentar 6.4: Aturan Soal Lebih Kuat daripada Kebiasaan Implementasi

Implementasi algoritma graf di buku atau pustaka dapat berbeda dalam menangani simpul duplikat, pembaruan cost, dan closed set. Untuk ujian, ikuti instruksi eksplisit soal. Misalnya soal 2025 menyatakan bahwa simpul yang sudah pernah masuk simpul ekspan tidak ditambahkan lagi ke simpul hidup, kecuali yang sebelumnya sudah ada di simpul hidup.

6.4.3 *Uninformed Search*

Inti Konsep.

Uninformed search tidak memakai informasi perkiraan jarak ke tujuan; keputusan ekspansi hanya bergantung pada struktur pencarian atau biaya aktual yang sudah ditempuh.

BFS, DFS, DLS, IDS, dan UCS termasuk *uninformed search*. Disebut *blind search* karena algoritma tidak mengetahui arah mana yang tampak lebih dekat ke tujuan. Perbedaan antaralgoritma terletak pada cara frontier dikelola.

Algoritma	Prioritas Ekspansi	Makna dan Kondisi Penggunaan
BFS	Kedalaman terkecil; frontier sebagai <i>queue</i> .	Cocok untuk graf tidak berbobot jika yang dicari adalah jumlah langkah minimum. Tidak mempertimbangkan bobot sisi.
DFS	Simpul terdalam; frontier sebagai <i>stack</i> .	Hemat memori dibanding BFS, tetapi dapat tersesat pada cabang dalam dan tidak menjamin solusi terpendek.
DLS	DFS dengan batas kedalaman.	Menghindari DFS masuk terlalu dalam, tetapi gagal jika solusi berada di bawah batas kedalaman.
IDS	DLS berulang dengan batas meningkat.	Menggabungkan kelebihan BFS untuk kedalaman solusi dan penggunaan memori seperti DFS, tetapi mengulang ekspansi pada level dangkal.
UCS	Biaya aktual terkecil $g(n)$.	Cocok untuk graf berbobot non-negatif dan menjamin lintasan optimal jika biaya sisi tidak negatif.

Tabel 8: Perbandingan algoritma uninformed search

6.4.4 Uniform Cost Search

Inti Konsep.

UCS selalu mengekspansi simpul hidup dengan biaya aktual paling kecil dari simpul awal.

Pada UCS, nilai evaluasi simpul n adalah $f(n) = g(n)$. Nilai $g(n)$ tidak dihitung dari estimasi, melainkan dari jumlah bobot sisi yang benar-benar dilalui dari simpul awal ke simpul n . Karena itu, UCS dapat dipahami sebagai generalisasi BFS untuk graf berbobot: jika semua sisi memiliki bobot sama, urutan UCS akan menyerupai BFS; jika bobot sisi berbeda, UCS lebih tepat karena mengutamakan lintasan termurah, bukan lintasan terdangkal.

Dalam tabel UAS, setiap simpul hidup biasanya ditulis bersama jalurnya atau nama simpulnya dan nilai $f(n)$. Jika terdapat dua jalur menuju simpul yang sama, periksa apakah jalur baru memberi $g(n)$ lebih kecil. Untuk pencarian graf yang benar, jalur dengan biaya lebih kecil seharusnya menggantikan jalur yang lebih mahal di frontier. Namun, bila soal memberikan aturan khusus tentang simpul yang sudah masuk simpul hidup atau simpul ekspansi, aturan tersebut harus diikuti.

6.4.5 Informed Search dan Heuristik

Inti Konsep.

Informed search memakai heuristik $h(n)$ untuk memperkirakan jarak atau biaya tersisa dari simpul n ke tujuan.

Heuristik adalah fungsi estimasi. Pada soal route planning, heuristik dapat diberikan langsung dalam tabel, atau didefinisikan sebagai jarak tertentu, misalnya Manhattan distance, Euclidean distance, atau banyaknya sisi minimal menuju tujuan. Heuristik yang baik membantu algoritma mengarah ke tujuan dengan lebih sedikit iterasi, tetapi heuristik yang buruk dapat menyedatkan pencarian.

Perlu dibedakan antara $h(n)$ dan $h^*(n)$. Nilai $h(n)$ adalah estimasi yang diberikan atau dihitung oleh soal. Nilai $h^*(n)$ adalah biaya optimal sebenarnya dari simpul n ke tujuan. Karena $h^*(n)$ adalah nilai sebenarnya, ia biasanya harus dihitung dari graf, misalnya dengan mencari lintasan termurah dari simpul tersebut ke tujuan.

6.4.6 Greedy Best-First Search

Inti Konsep.

Greedy Best-First Search memilih simpul yang tampak paling dekat ke tujuan berdasarkan nilai heuristik terkecil.

Greedy Best-First Search memakai $f(n) = h(n)$. Algoritma ini mengabaikan biaya yang sudah dikeluarkan dari simpul awal ke simpul n . Akibatnya, Greedy dapat menemukan tujuan dengan sedikit iterasi, tetapi tidak menjamin lintasan termurah. Dalam pola soal UAS, Greedy sering dibandingkan dengan UCS dan A*: Greedy dapat menang dari sisi jumlah iterasi, tetapi kalah dari sisi total cost.

Intuisi Greedy sederhana: jika tujuan berada di “arah” tertentu menurut heuristik, algoritma akan terus mendekati arah itu. Masalahnya, lintasan yang tampak dekat bisa melalui sisi yang mahal. Karena itu, Greedy cocok dipahami sebagai strategi agresif menuju tujuan, bukan strategi optimal untuk biaya lintasan.

6.4.7 A*

Inti Konsep.

A* menyeimbangkan biaya yang sudah ditempuh dan estimasi biaya tersisa dengan fungsi $f(n) = g(n) + h(n)$.

A* menggabungkan kekuatan UCS dan Greedy Best-First Search. Komponen $g(n)$ menjaga agar algoritma tidak mengabaikan biaya aktual yang sudah dikeluarkan, sedangkan $h(n)$ mengarahkan pencarian menuju tujuan. Jika $h(n) = 0$ untuk semua simpul, A* berubah menjadi UCS. Jika $g(n)$ diabaikan, perilakunya menjadi Greedy Best-First Search.

Kelebihan A* bergantung pada kualitas heuristik. Heuristik yang terlalu kecil tetap aman tetapi kurang informatif, sehingga A* mendekati UCS dan dapat mengeksplorasi banyak simpul. Heuristik yang terlalu besar dapat membuat A* memilih lintasan yang tampak murah tetapi sebenarnya bukan lintasan optimal.

6.4.8 Admissible dan Consistent Heuristic

Inti Konsep.

Heuristik admissible tidak pernah melebihi-lebihkan biaya optimal sebenarnya menuju tujuan; heuristik consistent juga memenuhi ketaksamaan segitiga pada setiap sisi.

Sebuah heuristik disebut admissible jika untuk setiap simpul n , estimasi $h(n)$ tidak lebih besar daripada biaya optimal sebenarnya $h^*(n)$. Dengan kata lain, heuristik boleh terlalu optimistis, tetapi tidak boleh terlalu pesimistis. Jika suatu simpul memiliki $h(n) > h^*(n)$, maka heuristik pada simpul tersebut tidak admissible.

Heuristik consistent atau monotone memenuhi syarat lebih kuat: untuk setiap sisi (n, n') , nilai heuristik dari n tidak boleh lebih besar daripada biaya dari n ke n' ditambah heuristik dari n' . Secara intuitif, estimasi dari suatu simpul tidak boleh turun terlalu tajam setelah mengambil satu langkah. Konsistensi penting pada pencarian graf karena membantu menjamin bahwa setelah suatu simpul diekspansi, biaya terbaiknya sudah final.

Komentar 6.5: Hubungan Admissible dan Optimalitas A*

Untuk *tree search*, admissibility cukup untuk menjamin A* optimal. Untuk *graph search* dengan closed set, syarat consistent lebih kuat dan lazim digunakan agar simpul yang sudah diekspansi tidak perlu dibuka kembali. Dalam ujian IF2221, soal yang tersedia lebih

sering meminta uji admissibility eksplisit daripada pembuktian konsistensi.

6.5 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Simpul awal s	Titik mulai pencarian. Semua nilai $g(n)$ dihitung relatif terhadap simpul ini.
Simpul tujuan t	Titik akhir yang dicari. Pencarian pada pola soal biasanya berhenti ketika t menjadi simpul ekspansi.
Sisi (u, v)	Perpindahan dari simpul u ke simpul v . Pada graf berarah, arah sisi harus dipatuhi.
Bobot sisi $c(u, v)$	Biaya berpindah dari u ke v , misalnya jarak, waktu, atau cost abstrak.
$g(n)$	Biaya aktual lintasan dari simpul awal ke simpul n .
$h(n)$	Estimasi biaya dari simpul n ke tujuan.
$h^*(n)$	Biaya optimal sebenarnya dari simpul n ke tujuan. Dipakai untuk memeriksa admissibility.
$f(n)$	Fungsi evaluasi untuk menentukan prioritas simpul hidup berikutnya.
Frontier/simpul hidup	Kumpulan kandidat simpul yang sudah ditemukan tetapi belum diekspansi.
Simpul ekspansi	Simpul yang dipilih pada iterasi berjalan untuk diperiksa dan dikembangkan tetangganya.
Closed set	Kumpulan simpul yang sudah diekspansi. Dalam soal, simpul ini biasanya tidak dimasukkan lagi ke simpul hidup.
Parent pointer	Informasi pendahulu setiap simpul yang dipakai untuk merekonstruksi jalur hasil.
Tie-breaking	Aturan pemilihan saat beberapa simpul memiliki nilai prioritas sama, misalnya urutan abjad.

Tabel 9: Komponen kunci dalam materi Route Planning

6.6 Algoritma dan Rumus Penting

6.6.1 Biaya Lintasan

Makna Rumus.

Biaya lintasan adalah jumlah bobot sisi yang dilalui dari simpul awal ke suatu simpul.

$$g(n) = \sum_{(u,v) \in \text{path}(s,n)} c(u,v) \quad (15)$$

dengan:

- s : simpul awal.
- n : simpul yang sedang dinilai.
- $\text{path}(s, n)$: lintasan dari s ke n .
- $c(u, v)$: biaya sisi dari u ke v .

Rumus ini dipakai pada UCS dan A*. Pada Greedy Best-First Search, $g(n)$ tetap dapat ditulis untuk menghitung jarak jalur akhir, tetapi tidak dipakai sebagai prioritas ekspansi.

6.6.2 Fungsi Evaluasi UCS, Greedy Best-First Search, dan A*

Makna Rumus.

Perbedaan UCS, Greedy Best-First Search, dan A* terletak pada fungsi evaluasi $f(n)$ yang digunakan untuk memilih simpul hidup berikutnya.

$$f_{\text{UCS}}(n) = g(n), \quad (16)$$

$$f_{\text{Greedy}}(n) = h(n), \quad (17)$$

$$f_{\text{A}^*}(n) = g(n) + h(n). \quad (18)$$

dengan:

- $g(n)$: biaya aktual dari simpul awal ke n .
- $h(n)$: estimasi biaya dari n ke tujuan.
- $f(n)$: prioritas ekspansi; simpul dengan $f(n)$ terkecil dipilih lebih dulu.

Aturan pemilihan simpul berikutnya dapat ditulis:

$$n_{\text{next}} = \arg \min_{n \in \text{frontier}} f(n) \quad (19)$$

Jika ada beberapa simpul dengan nilai $f(n)$ sama, gunakan aturan tie-breaking pada soal. Bila tidak disebutkan, pilihan harus dinyatakan secara eksplisit agar tabel dapat diikuti.

6.6.3 Admissibility Heuristic

Makna Rumus.

Admissibility memeriksa apakah heuristik tidak pernah melebihi-lebihkan biaya optimal sebenarnya menuju tujuan.

$$0 \leq h(n) \leq h^*(n) \quad (20)$$

dengan:

- $h(n)$: heuristik yang diberikan atau dihitung untuk simpul n .
- $h^*(n)$: biaya optimal sebenarnya dari simpul n ke tujuan.

Untuk mengecek admissibility pada soal, lakukan langkah berikut:

1. Untuk setiap simpul n , cari biaya lintasan termurah sebenarnya dari n ke tujuan.
2. Bandingkan nilai tersebut dengan heuristik $h(n)$.
3. Jika $h(n) \leq h^*(n)$, heuristik pada simpul n admissible.
4. Jika $h(n) > h^*(n)$, heuristik pada simpul n tidak admissible.

6.6.4 Consistency Heuristic

Makna Rumus.

Consistency memastikan estimasi heuristik tidak melanggar ketaksamaan segitiga pada setiap sisi.

$$h(n) \leq c(n, n') + h(n') \quad (21)$$

dengan:

- n' : tetangga dari simpul n .
- $c(n, n')$: biaya sisi dari n ke n' .
- $h(n)$ dan $h(n')$: estimasi biaya masing-masing simpul menuju tujuan.

Setiap heuristik consistent pasti admissible jika $h(t) = 0$ pada simpul tujuan. Kebalikannya tidak selalu benar: heuristik dapat admissible tetapi tidak consistent.

6.6.5 Pseudocode Pencarian Best-First Generik

Tujuan Algoritma.

Pseudocode ini menyatukan UCS, Greedy Best-First Search, dan A* dengan mengganti fungsi evaluasi $f(n)$.

Listing 2: Pseudocode pencarian best-first untuk route planning

```
1 Input:
2   graph, start, goal
3   evaluation function f(n)
4 Output:
5   path from start to goal, total cost, iteration table
6
7 1. frontier <- priority queue ordered by f(n)
8 2. insert start into frontier with g(start) = 0
9 3. parent[start] <- None
10 4. closed <- empty set
11 5. iteration <- 0
12
13 6. while frontier is not empty:
14 7.   n <- pop node with smallest f(n) from frontier
15 8.   if n is in closed:
16 9.     continue
17 10.  iteration <- iteration + 1
18 11.  record n as expanded node and record current frontier
19 12.  if n == goal:
20 13.    return reconstruct_path(parent, goal), g(goal), iteration
21 14.  add n to closed
22 15.  for each neighbor m of n:
23 16.    if m is in closed:
24 17.      continue
25 18.    tentative_g <- g(n) + cost(n, m)
26 19.    if m not in frontier or tentative_g is better than g(m):
27 20.      g(m) <- tentative_g
28 21.      parent[m] <- n
29 22.      insert or update m in frontier using f(m)
30
31 23. return "Tidak ada solusi"
```

Untuk UCS, gunakan $f(n) = g(n)$. Untuk Greedy Best-First Search, gunakan $f(n) = h(n)$. Untuk A*, gunakan $f(n) = g(n) + h(n)$. Pada jawaban manual UAS, struktur prioritas ini diterjemahkan menjadi tabel iterasi: baris ke- i berisi simpul ekspan pada iterasi tersebut dan daftar simpul hidup setelah ekspansi.

6.6.6 Template Tabel Iterasi UAS

Tujuan Tabel.

Tabel iterasi membuat proses pemilihan simpul dapat diperiksa langkah demi langkah.

Iterasi	Simpul Ekspan	Simpul Hidup dan Nilai Evaluasi
1	A	$B : f(B) = \dots, C : f(C) = \dots$
2	$\dots$	$\dots$
k	Tujuan	Pencarian berhenti karena simpul tujuan menjadi simpul ekspan.

Tabel 10: Template tabel iterasi route planning untuk UAS

Agar jawaban lengkap, setelah tabel tuliskan:

- jalur hasil, misalnya $A \rightarrow C \rightarrow D \rightarrow F$;
- jarak atau cost total, yaitu jumlah bobot sisi pada jalur hasil;
- banyak iterasi, dihitung dari jumlah baris ekspansi sampai tujuan menjadi simpul ekspan;
- aturan tie-breaking yang dipakai jika ada nilai sama;
- catatan apabila tidak ada solusi.

6.7 Kesalahan Umum dan Edge Case

- **Berhenti terlalu cepat:** tujuan baru muncul di frontier belum berarti pencarian selesai. Berhenti saat tujuan menjadi simpul ekspan.
- **Mencampur rumus evaluasi:** UCS memakai $g(n)$, Greedy memakai $h(n)$, A* memakai $g(n) + h(n)$.
- **Menghitung $g(n)$ dari simpul yang salah:** $g(n)$ selalu biaya dari simpul awal ke n , bukan dari simpul ekspan saat ini saja.
- **Menganggap Greedy selalu optimal:** Greedy dapat memilih jalur yang tampak dekat tetapi mahal.
- **Mengabaikan arah sisi:** pada graf berarah, sisi hanya boleh dilalui sesuai arah panah.
- **Mengabaikan tie-breaking:** jika nilai $f(n)$ sama, pilihan simpul harus mengikuti aturan soal.
- **Tidak memperbarui jalur:** jika simpul yang sama ditemukan melalui jalur lebih murah dan aturan soal mengizinkan, parent dan $g(n)$ harus diperbarui.
- **Salah uji admissibility:** yang dibandingkan dengan $h(n)$ adalah biaya optimal sebenarnya dari simpul itu ke tujuan, bukan biaya dari simpul awal ke simpul itu.

6.8 Checklist Pemahaman

6.8.1 Submateri yang Telah Dibahas

- ☐ Saya dapat memodelkan persoalan route planning sebagai graf dengan simpul, sisi, bobot, simpul awal, dan simpul tujuan.
- ☐ Saya dapat membedakan simpul hidup, simpul ekspan, closed set, dan parent pointer.
- ☐ Saya dapat menjelaskan perbedaan BFS, DFS, DLS, IDS, dan UCS.
- ☐ Saya dapat menghitung $g(n)$ sebagai biaya aktual lintasan dari simpul awal.
- ☐ Saya dapat menghitung $f(n) = g(n)$ untuk UCS.
- ☐ Saya dapat menghitung $f(n) = h(n)$ untuk Greedy Best-First Search.
- ☐ Saya dapat menghitung $f(n) = g(n) + h(n)$ untuk A*.
- ☐ Saya dapat membuat tabel iterasi UCS, Greedy Best-First Search, dan A* sampai simpul tujuan menjadi simpul ekspan.
- ☐ Saya dapat menentukan jalur hasil, total cost, dan banyak iterasi dari tabel pencarian.
- ☐ Saya dapat menguji apakah heuristik admissible dengan membandingkan $h(n)$ dan $h^*(n)$.
- ☐ Saya dapat menjelaskan mengapa Greedy tidak selalu optimal, UCS optimal untuk biaya non-negatif, dan A* optimal jika heuristik memenuhi syarat.
- ☐ Saya dapat menjelaskan hubungan Route Planning dengan Backtracking, Branch and Bound, dan Program Dinamis.

6.8.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Tabel iterasi UCS, Greedy Best-First Search, dan A*; perhitungan $g(n)$, $h(n)$, dan $f(n)$; kondisi berhenti saat tujuan menjadi simpul ekspan; jalur hasil; jarak total; banyak iterasi; dan uji admissibility heuristik.
Cukup Pahami Konsep	BFS, DFS, DLS, IDS, consistency heuristic, tree search versus graph search, parent pointer, closed set, dan hubungan route planning dengan materi lain.
Sudah dikuasai	Belum ditentukan.

7 String Matching

String Matching membahas pencarian kemunculan suatu pola (*pattern*) di dalam teks. Secara formal, diberikan teks T sepanjang n dan pattern P sepanjang m , persoalan string matching mencari indeks i sedemikian sehingga substring $T[i..i + m - 1]$ sama dengan $P[0..m - 1]$. Materi ini penting karena memperlihatkan bagaimana informasi dari pattern dapat dipakai untuk menghindari perbandingan karakter yang berulang.

Fokus UAS adalah kemampuan menjalankan algoritma secara manual: menghitung preprocessing KMP dan Boyer-Moore, menggambarkan proses pencocokan karakter demi karakter, menghitung jumlah perbandingan, dan menjelaskan algoritma mana yang lebih efisien untuk pasangan teks-pattern tertentu.

Komentar 7.1: Keterbatasan Sumber Utama

NotebookLM yang diberikan dapat diakses, tetapi jawaban yang diperoleh untuk String Matching hanya memuat outline global: Brute Force, KMP dengan fungsi pinggiran, Boyer-Moore dengan *last occurrence function*, dan kompleksitas Brute Force serta KMP. Detail prosedural seperti cara menulis tabel perbandingan, aturan hitung operasi karakter, dan edge case dilengkapi menggunakan pengetahuan umum yang relevan serta pola soal UAS 2022–2025.

7.1 Outline Fundamental Konsep Materi

1. Model persoalan string matching
 - (a) Teks T sepanjang n .
 - (b) Pattern P sepanjang m .
 - (c) Alfabet karakter.
 - (d) Indeks awal kecocokan.
 - (e) Kemunculan pertama, semua kemunculan, dan kasus tidak ditemukan.
2. Terminologi string
 - (a) Prefix: awalan suatu string.
 - (b) Suffix: akhiran suatu string.
 - (c) Proper prefix dan proper suffix.
 - (d) Border: prefix yang sekaligus suffix.
 - (e) Shift atau pergeseran pattern.
 - (f) Perbandingan karakter sebagai satuan operasi manual.
3. Algoritma Brute Force
 - (a) Mencoba pattern pada setiap posisi teks.
 - (b) Mencocokkan karakter kiri ke kanan.
 - (c) Menggeser pattern satu posisi setelah mismatch.
 - (d) Kasus terburuk $O(mn)$.
4. Algoritma Knuth-Morris-Pratt (KMP)
 - (a) Preprocessing pattern.
 - (b) Fungsi pinggiran atau border function $b(k)$.
 - (c) Pergeseran berdasarkan prefix-suffix pattern.
 - (d) Kompleksitas $O(m + n)$.
5. Algoritma Boyer-Moore
 - (a) *Looking-glass technique*: perbandingan dari kanan pattern.
 - (b) *Character-jump technique*: lompatan berdasarkan karakter mismatch.
 - (c) *Last occurrence function* $L(x)$.
 - (d) Pergeseran $\max(1, j - L(x))$.
6. Pola pengerjaan UAS
 - (a) Membuat tabel border function KMP.

- (b) Membuat tabel last occurrence Boyer-Moore.
- (c) Menggambar alignment pattern terhadap teks.
- (d) Menghitung jumlah perbandingan karakter.
- (e) Membandingkan algoritma berdasarkan data soal, bukan hafalan umum.

7.2 Pentingnya Materi dan Relevansinya

String matching adalah dasar dari banyak sistem pencarian teks. Editor teks memakai string matching untuk fitur *find*. Mesin pencari, sistem indeks dokumen, bioinformatika, deteksi pola DNA, analisis log, dan pemrosesan bahasa alami juga membutuhkan gagasan yang sama: menemukan pola dalam rangkaian simbol. Dalam konteks Strategi Algoritma, string matching memperlihatkan bahwa perbaikan algoritma tidak selalu berasal dari struktur data besar; kadang perbaikan besar muncul dari memanfaatkan struktur internal input, yaitu prefix, suffix, dan posisi kemunculan karakter dalam pattern.

Komentar 7.2: Relevansi Praktis

Sumber utama menyebut aplikasi string matching pada editor teks, web search engine, analisis citra, dan bioinformatika. Catatan ini menambahkan penjelasan praktis secara umum, tetapi tetap membatasi pembahasan teknis pada algoritma yang diuji: Brute Force, KMP, dan Boyer-Moore.

7.2.1 Dependensi dan Hubungan dengan Materi Lain

String Matching menjadi dasar bagi String Matching with Regex. Pada string matching biasa, pattern adalah string literal yang harus cocok persis. Pada regex, pattern menjadi bahasa pola yang dapat menyatakan pilihan karakter, pengulangan, batas kata, dan struktur yang lebih fleksibel. Secara konseptual, keduanya sama-sama membahas pencocokan pola terhadap teks, tetapi regex memperluas bentuk pattern.

Materi ini juga berhubungan dengan analisis kompleksitas pada Teori P/NP/NPC dalam arti lebih umum: kita tidak hanya ingin algoritma benar, tetapi juga ingin tahu jumlah operasi yang dilakukan. Soal UAS string matching hampir selalu meminta jumlah perbandingan karakter, sehingga kompleksitas bukan konsep abstrak, melainkan langsung terlihat pada tabel pencocokan.

7.3 Pola Soal UAS Sebelumnya dan Prioritas

Komentar 7.3: Pola dari Soal 2022–2025

String matching muncul konsisten pada dokumen soal yang diperiksa. Soal 2022 meminta pemilihan metode yang lebih baik, proses pencocokan, dan jumlah perbandingan. Solusi 2023 meminta contoh kasus KMP sama buruknya dengan Brute Force dan penelusuran Boyer-Moore. Soal 2024 meminta tabel border function KMP, tabel last occurrence Boyer-Moore, ilustrasi langkah, dan total operasi. Soal 2025 memakai konteks DNA dan meminta KMP, Boyer-Moore, serta algoritma dengan perbandingan paling sedikit.

Pola Soal	Yang Biasanya Diminta	Kemampuan Wajib
KMP pada DNA atau string pendek	Hitung fungsi pinggiran, lalu gambarkan pencocokan sampai ditemukan atau gagal.	Menentukan border setiap prefix pattern dan melakukan fallback tanpa mengulang perbandingan yang tidak perlu.
Boyer-Moore	Hitung last occurrence, cocokkan dari kanan, lalu hitung jumlah perbandingan.	Menentukan $L(x)$, posisi mismatch j , dan pergeseran $\max(1, j - L(x))$.
Perbandingan algoritma	Menjawab algoritma mana yang lebih baik untuk teks-pattern tertentu.	Menilai berdasarkan jumlah perbandingan aktual, bukan sekadar klaim bahwa satu algoritma selalu lebih baik.
Brute Force versus KMP	Memberi contoh atau menghitung kasus ketika perbandingan sama banyak.	Memahami bahwa KMP unggul karena preprocessing, tetapi pada input tertentu jumlah perbandingannya dapat sama dengan Brute Force.

Tabel 11: Pola soal UAS String Matching

7.4 Penjelasan Konsep Fundamental

7.4.1 Definisi Teks, Pattern, dan Indeks Kecocokan

Inti Konsep.

String matching mencari posisi di dalam teks tempat seluruh karakter pattern cocok secara berurutan.

Misalkan $T = T[0]T[1] \dots T[n-1]$ dan $P = P[0]P[1] \dots P[m-1]$. Pattern muncul pada posisi i jika untuk semua j dengan $0 \leq j < m$, berlaku $T[i+j] = P[j]$. Posisi i yang mungkin hanya dari 0 sampai $n-m$. Jika $m > n$, pattern tidak mungkin muncul karena pattern lebih panjang daripada teks.

Pada pengerjaan manual, setiap perbandingan karakter harus dihitung satu per satu. Jika pada suatu alignment pattern terjadi mismatch pada perbandingan pertama, tetap dihitung satu operasi perbandingan. Jika seluruh m karakter cocok, jumlah operasi pada alignment tersebut adalah m .

7.4.2 Prefix, Suffix, dan Border

Inti Konsep.

Border adalah substring yang menjadi prefix sekaligus suffix; KMP memakai border untuk menentukan seberapa jauh pattern dapat digeser tanpa kehilangan kemungkinan match.

Prefix adalah awalan string. Suffix adalah akhiran string. Misalnya untuk string ABABA, prefix-nya mencakup A, AB, ABA, dan seterusnya; suffix-nya mencakup A, BA, ABA, dan seterusnya. Border adalah prefix yang juga suffix, biasanya tidak termasuk seluruh string itu sendiri jika yang dimaksud adalah proper border.

KMP memanfaatkan border karena saat sebagian pattern sudah cocok lalu terjadi mismatch, bagian yang sudah cocok tidak perlu dibandingkan dari awal. Jika suffix dari bagian yang cocok sama dengan prefix pattern, maka prefix tersebut dapat dipertahankan sebagai kecocokan awal pada alignment berikutnya.

7.4.3 Brute Force

Inti Konsep.

Brute Force mencoba semua posisi awal yang mungkin dan mencocokkan pattern dari kiri ke kanan.

Brute Force adalah algoritma paling langsung. Pada setiap posisi i dalam teks, algoritma membandingkan $P[0]$ dengan $T[i]$, lalu $P[1]$ dengan $T[i + 1]$, dan seterusnya sampai terjadi mismatch atau seluruh pattern cocok. Jika mismatch terjadi, pattern digeser satu posisi ke kanan.

Kelebihan Brute Force adalah mudah dipahami dan tidak memerlukan preprocessing. Kekurangannya, algoritma ini dapat membandingkan ulang karakter teks yang sebenarnya sudah diketahui. Kasus buruk muncul ketika pattern hampir selalu cocok panjang di banyak posisi, lalu gagal di karakter terakhir.

7.4.4 Knuth-Morris-Pratt

Inti Konsep.

KMP mencocokkan dari kiri ke kanan seperti Brute Force, tetapi saat mismatch terjadi, pattern tidak selalu kembali ke awal karena ada informasi border.

KMP memiliki dua fase. Fase pertama adalah preprocessing pattern untuk menghitung fungsi pinggiran $b(k)$. Fase kedua adalah pencocokan teks menggunakan fungsi tersebut. Nilai $b(k)$ menyatakan panjang border terbesar untuk prefix pattern yang berakhir di indeks k . Jika telah cocok j karakter dan mismatch terjadi, KMP tidak menggeser satu per satu, tetapi mengatur ulang j ke nilai border yang sesuai.

Kesalahan umum pada KMP adalah menghitung border terhadap teks. Border function hanya dihitung dari pattern, bukan dari teks. Teks baru digunakan pada fase pencocokan.

7.4.5 Boyer-Moore

Inti Konsep.

Boyer-Moore mencocokkan pattern dari kanan ke kiri dan dapat melompat lebih jauh saat karakter mismatch tidak cocok dengan struktur pattern.

Versi yang biasa diuji pada IF2221 menekankan *looking-glass technique* dan *character-jump technique*. *Looking-glass* berarti perbandingan dimulai dari karakter paling kanan pattern. Jika terjadi mismatch, algoritma melihat karakter teks yang menyebabkan mismatch, lalu memakai *last occurrence function* untuk menentukan pergeseran.

Jika karakter mismatch tidak muncul di pattern, pattern dapat digeser melewati karakter tersebut. Jika karakter mismatch muncul di pattern, pattern digeser agar kemunculan terakhir karakter itu sejajar dengan posisi mismatch. Jika perhitungan menghasilkan pergeseran nol atau negatif, gunakan pergeseran minimum satu posisi.

7.5 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
T	Teks tempat pattern dicari.
P	Pattern yang dicari di dalam teks.
n	Panjang teks.
m	Panjang pattern.
Alignment	Posisi pattern relatif terhadap teks saat pencocokan dilakukan.
Prefix	Awalan string. Dipakai untuk memahami border KMP.
Suffix	Akhiran string. Dipakai untuk memahami border KMP.
Border	Prefix yang juga suffix pada substring pattern.
$b(k)$	Fungsi pinggiran KMP untuk prefix pattern yang berakhir pada indeks k .
$L(x)$	Indeks kemunculan terakhir karakter x di pattern pada Boyer-Moore, atau -1 jika tidak muncul.
Jumlah perbandingan	Banyaknya operasi membandingkan satu karakter teks dengan satu karakter pattern.

Tabel 12: Komponen kunci dalam materi String Matching

7.6 Algoritma dan Rumus Penting

7.6.1 Brute Force String Matching

Tujuan Algoritma.

Brute Force mengembalikan indeks awal kemunculan pattern atau -1 jika pattern tidak ditemukan.

Listing 3: Pseudocode Brute Force String Matching

```

1 Input: text T with length n, pattern P with length m
2 Output: first index where P occurs in T, or -1
3
4 for i = 0 to n - m:
5     j = 0
6     while j < m and T[i + j] == P[j]:
7         j = j + 1
8     if j == m:
9         return i
10 return -1

```

Posisi awal yang diperiksa adalah:

$$0 \leq i \leq n - m \quad (22)$$

Kasus terburuk jumlah perbandingan Brute Force:

$$C_{\text{brute}} = m(n - m + 1) = O(mn) \quad (23)$$

dengan $n - m + 1$ adalah jumlah alignment yang mungkin dan m adalah jumlah karakter pattern yang dapat dibandingkan pada setiap alignment.

7.6.2 Fungsi Pinggiran KMP

Makna Rumus.

Fungsi pinggiran menyimpan panjang border terbesar sehingga KMP tahu ke mana harus mundur saat mismatch.

$$b(k) = \max\{r \mid P[0..r-1] = P[k-r+1..k]\} \quad (24)$$

dengan:

- k : indeks terakhir prefix pattern yang sedang dianalisis.
- r : panjang border.
- $P[0..r-1]$: prefix sepanjang r .
- $P[k-r+1..k]$: suffix sepanjang r dari $P[0..k]$.

Saat mismatch terjadi setelah j karakter pattern cocok, fallback KMP dapat ditulis:

$$j \leftarrow b(j-1) \quad (25)$$

Kompleksitas total KMP:

$$T_{\text{KMP}}(n, m) = O(m + n) \quad (26)$$

7.6.3 Pseudocode KMP

Tujuan Algoritma.

KMP mencari pattern dengan memanfaatkan border sehingga karakter teks tidak perlu dibandingkan mundur berulang kali.

Listing 4: Pseudocode KMP

```
1 Input: text T with length n, pattern P with length m
2 Output: first index where P occurs in T, or -1
3
4 b = compute_border_function(P)
5 i = 0          # index in T
6 j = 0          # index in P
7
8 while i < n:
9     if T[i] == P[j]:
10         i = i + 1
11         j = j + 1
12         if j == m:
13             return i - m
14     else:
15         if j > 0:
16             j = b[j - 1]
17         else:
18             i = i + 1
19
20 return -1
```

Untuk menghitung jumlah perbandingan pada KMP, setiap eksekusi pemeriksaan $T[i] == P[j]$ dihitung satu kali. Perubahan j karena border bukan perbandingan karakter baru.

7.6.4 Last Occurrence Boyer-Moore

Makna Rumus.

Last occurrence menyimpan posisi paling kanan suatu karakter di pattern agar Boyer-Moore dapat melompat saat mismatch.

$$L(x) = \begin{cases} \max\{i \mid P[i] = x\}, & \text{jika } x \text{ muncul di } P, \\ -1, & \text{jika } x \text{ tidak muncul di } P. \end{cases} \quad (27)$$

Jika mismatch terjadi pada posisi pattern j terhadap karakter teks x , pergeseran *character-jump* adalah:

$$\text{shift} = \max(1, j - L(x)) \quad (28)$$

7.6.5 Pseudocode Boyer-Moore dengan Character-Jump

Tujuan Algoritma.

Boyer-Moore mencari pattern dari kanan ke kiri dan memakai last occurrence untuk menentukan lompatan pattern.

Listing 5: Pseudocode Boyer-Moore dengan character-jump

```
1 Input: text T with length n, pattern P with length m
2 Output: first index where P occurs in T, or -1
3
4 L = last_occurrence_table(P)
5 i = m - 1          # index in T aligned with last char of P
6 j = m - 1          # index in P
7
8 while i < n:
9     if T[i] == P[j]:
10         if j == 0:
11             return i
12         i = i - 1
13         j = j - 1
14     else:
15         x = T[i]
16         shift = m - min(j, 1 + L[x])
17         i = i + shift
18         j = m - 1
19
20 return -1
```

Pada jawaban manual, lebih aman menuliskan alignment pattern dan menghitung langsung perbandingan dari kanan ke kiri. Setelah mismatch, gunakan rumus shift yang diajarkan atau yang dinyatakan di soal. Jika rumus dalam salindia memakai notasi berbeda, pertahankan makna yang sama: sejajarkan kemunculan terakhir karakter mismatch di pattern atau geser melewati karakter itu jika tidak muncul.

7.7 Cara Menjawab Soal Manual

7.7.1 Format Jawaban KMP

1. Tulis teks T dan pattern P dengan indeks.
2. Hitung tabel border function $b(k)$ untuk setiap indeks pattern.
3. Mulai pencocokan dari kiri pattern.

4. Setiap kali cocok, lanjut ke karakter berikutnya.
5. Setiap kali mismatch, hitung fallback menggunakan $b(j - 1)$.
6. Hitung satu operasi untuk setiap perbandingan karakter.
7. Jika pattern ditemukan, tulis indeks awal, jumlah perbandingan, dan alignment akhir.

7.7.2 Format Jawaban Boyer-Moore

1. Tulis tabel $L(x)$ untuk semua karakter yang relevan, terutama karakter dalam pattern dan karakter mismatch pada teks.
2. Letakkan pattern di bawah teks pada alignment awal.
3. Bandingkan dari karakter paling kanan pattern.
4. Jika mismatch pada pattern indeks j dengan karakter teks x , hitung shift.
5. Geser pattern dan ulangi.
6. Hitung satu operasi untuk setiap perbandingan karakter.
7. Jika pattern ditemukan, tulis indeks awal, jumlah perbandingan, dan alignment akhir.

7.8 Kesalahan Umum dan Edge Case

- **Menghitung border dari teks:** border function KMP hanya berasal dari pattern.
- **Menghitung jumlah shift sebagai jumlah perbandingan:** yang dihitung adalah perbandingan karakter, bukan jumlah pergeseran pattern.
- **Boyer-Moore dibandingkan dari kiri:** versi Boyer-Moore pada materi ini memakai perbandingan dari kanan pattern.
- **Lupa karakter yang tidak muncul:** pada Boyer-Moore, jika x tidak muncul dalam pattern, $L(x) = -1$.
- **Menganggap KMP selalu lebih sedikit perbandingan dari Brute Force:** KMP memiliki jaminan kompleksitas linear, tetapi pada instance tertentu jumlah perbandingan aktual dapat sama.
- **Mengabaikan indeks:** kesalahan indeks menyebabkan border, shift, dan jumlah perbandingan menjadi salah.
- **Pattern lebih panjang dari teks:** hasil langsung tidak ditemukan.

7.9 Checklist Pemahaman

7.9.1 Submateri yang Telah Dibahas

- ☐ Saya dapat mendefinisikan teks, pattern, indeks kecocokan, prefix, suffix, dan border.
- ☐ Saya dapat menelusuri Brute Force dan menghitung jumlah perbandingan karakternya.
- ☐ Saya dapat menghitung fungsi pinggiran KMP untuk pattern yang diberikan.
- ☐ Saya dapat menjalankan KMP sampai pattern ditemukan atau teks habis.
- ☐ Saya dapat menghitung last occurrence function Boyer-Moore.

- ☐ Saya dapat menjalankan Boyer-Moore dari kanan ke kiri dan menentukan shift saat mismatch.
- ☐ Saya dapat membandingkan Brute Force, KMP, dan Boyer-Moore berdasarkan jumlah perbandingan pada instance soal.
- ☐ Saya dapat menjelaskan hubungan String Matching dengan Regex.

7.9.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Tabel border function KMP, tabel last occurrence Boyer-Moore, proses pencocokan karakter demi karakter, jumlah perbandingan, dan perbandingan hasil algoritma.
Cukup Pahami Konsep	Kompleksitas Brute Force, KMP, dan Boyer-Moore; aplikasi pada editor teks, search engine, analisis citra, dan bioinformatika.
Sudah dikuasai	Belum ditentukan.

8 String Matching with Regex

String Matching with Regex membahas pencocokan string menggunakan regular expression, yaitu notasi standar untuk mendeskripsikan pola barisan karakter. Fokus UAS adalah memahami notasi dasar regex, character class, quantifier, boundary matcher, dan penggunaan dasar regex di Python.

8.1 Outline Fundamental Konsep Materi

1. Definisi regex
 - (a) Pattern berbasis notasi
 - (b) Exact matching vs regex matching
2. Notasi dasar
 - (a) Character class
 - (b) Range
 - (c) Negasi
 - (d) Titik dan escape
3. Quantifier dan boundary
 - (a) `?`, `*`, `+`, dan bentuk jumlah tertentu
 - (b) `^` untuk awal, `$` untuk akhir, dan `\b` untuk batas kata
4. Regex di Python
 - (a) `re.compile`
 - (b) `search`
 - (c) `match`
 - (d) `findall`

8.2 Pentingnya Materi dan Relevansinya

8.2.1 Mengapa Materi Ini Penting

Regex penting karena banyak kebutuhan pencarian teks tidak cukup diselesaikan dengan exact matching. Regex memungkinkan pola seperti huruf kapital diikuti huruf kecil, digit sejumlah tertentu, alamat email, atau nomor telepon dinyatakan secara ringkas.

8.2.2 Implementasi Dasar

Implementasi dasar regex di Python dilakukan dengan membuat pattern, lalu menerapkannya pada string menggunakan `search`, `match`, atau `findall`.

8.2.3 Relevansi dengan Industri Saat Ini

Komentar 8.1: Keterbatasan Sumber

Sumber utama menampilkan contoh email, nomor telepon, dan ELIZA-like chatbot berbasis pattern matching. Catatan ini tidak menambahkan contoh di luar sumber tersebut.

8.2.4 Dependensi dan Hubungan dengan Materi Lain

Regex melanjutkan String Matching. Perbedaannya, pattern tidak harus literal, tetapi dapat berupa aturan yang mencakup banyak kemungkinan string.

8.3 Pola Soal UAS Sebelumnya dan Prioritas

Komentar 8.2: Pola dari Soal 2022–2025

Regex muncul pada semua sumber soal yang diperiksa. Bentuknya bergeser antara membaca regex yang diberikan, menentukan substring/kata yang cocok, menulis regex dari spesifikasi format, dan pada 2024 mengubah finite automata menjadi regex.

Prioritas utama adalah character class, negasi, quantifier, boundary, escape karakter khusus, dan hasil `findall`. Pada 2022, soal meminta menjelaskan pola `[\w\s]+ly\b` dan hasil `re.findall`. Pada solusi 2023, soal meminta semua string yang cocok dengan `[A-Za-z]{2}\d{4}`. Pada 2024, soal meminta regex dari finite automata dan menentukan kata yang cocok dengan regex `m[ae]n?[dsk]*[jtiudf]`. Pada 2025, soal meminta menulis regex untuk nama file gambar, nomor telepon, dan menentukan string yang cocok untuk regex dengan negasi.

- **Pola konstruksi:** tulis regex dari spesifikasi format file, nomor telepon, atau pola sederhana lain.
- **Pola evaluasi:** tentukan substring atau kata yang cocok dengan regex yang diberikan.
- **Pola notasi:** jelaskan arti character class, negasi, quantifier, boundary, dan escape.

Permasalahan yang Sering Muncul	Yang Biasanya Diminta	Fungsi/Komponen Wajib Dipahami
Membuat regex dari spesifikasi	Menulis pola untuk nama file, nomor telepon, kode, atau format teks sederhana.	Character class, range, quantifier, escape <code>\</code> , anchor <code>^</code> dan <code>\$</code> .
Mengevaluasi regex pada daftar kata/teks	Menentukan kata atau substring yang cocok dengan regex yang diberikan.	<code>[]</code> , <code>[^]</code> , <code>?</code> , <code>*</code> , <code>+</code> , <code>{m}</code> , <code>{m,n}</code> .
Regex di Python	Menjelaskan hasil <code>re.findall</code> atau perbedaan cara pencarian.	<code>findall</code> , <code>search</code> , <code>match</code> , boundary <code>\b</code> , whitespace <code>\s</code> .

Tabel 13: Permasalahan dan fungsi penting pada Regex

8.3.1 Formula Eksplisit yang Perlu dikuasai

Inti Formula.

Regex tidak memakai rumus numerik seperti DP atau B&B, tetapi memakai aturan pembentukan bahasa. Formula yang perlu dikuasai adalah makna operator regex terhadap himpunan string yang dicocokkan.

Character class:

$$\mathcal{L}([abc]) \equiv \{a, b, c\}, \quad (29)$$

$$\mathcal{L}([a-z]) \equiv \{a, b, c, \dots, z\}, \quad (30)$$

$$\mathcal{L}(\text{negasi } [abc]) \equiv \Sigma \setminus \{a, b, c\}. \quad (31)$$

Quantifier:

$$R? \equiv \epsilon \cup R, \quad (32)$$

$$R^* \equiv \bigcup_{k=0}^{\infty} R^k, \quad (33)$$

$$R^+ \equiv \bigcup_{k=1}^{\infty} R^k, \quad (34)$$

$$R\{m\} \equiv R^m, \quad (35)$$

$$R\{m, n\} \equiv \bigcup_{k=m}^n R^k. \quad (36)$$

Boundary matcher:

- `^`: cocok hanya pada awal string atau awal baris.
- `$`: cocok hanya pada akhir string atau akhir baris.
- `\b`: cocok pada batas kata.

Contoh pola file gambar yang sering muncul:

`\w+\.(gif|png|jpg|jpeg)`

Contoh pola nomor telepon +62-022-27204:

`\+\d{2}-\d{3}-\d{5}`

8.4 Penjelasan Konsep Fundamental

8.4.1 Definisi Regex

Inti Konsep.

Regular expression adalah notasi standar yang mendeskripsikan pola berupa urutan karakter atau string.

Regex digunakan untuk string matching yang lebih fleksibel daripada pencocokan persis. Pattern seperti `[A-Z][a-z]*` dapat menyatakan kata yang diawali huruf kapital dan diikuti nol atau banyak huruf kecil.

8.4.2 Character Class

Inti Konsep.

Character class menyatakan himpunan karakter yang boleh muncul pada satu posisi pattern.

Contoh `[abc]` cocok dengan karakter `a`, `b`, atau `c`. Range seperti `[a-z]` cocok dengan huruf kecil dari `a` sampai `z`. Negasi seperti `[^abc]` cocok dengan karakter selain `a`, `b`, dan `c`.

8.4.3 Quantifier dan Boundary Matcher

Inti Konsep.

Quantifier mengatur jumlah kemunculan pola, sedangkan boundary matcher mengatur posisi kemunculan pola.

Tanda `?` menyatakan pola boleh ada atau tidak. Tanda `*` menyatakan nol atau banyak kemunculan. Titik `.` menyatakan sembarang karakter, sedangkan titik literal perlu ditulis dengan escape. Boundary matcher seperti `^` dan `$` membatasi posisi awal dan akhir.

8.5 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
<code>[abc]</code>	Simple class: cocok dengan <code>a</code> , <code>b</code> , atau <code>c</code> .
<code>[a-z]</code>	Range karakter.
<code>[^abc]</code>	Negasi character class.
<code>.</code>	Metacharacter untuk sembarang karakter.
<code>?, *, +</code>	Quantifier untuk jumlah kemunculan pola.
<code>\d</code>	Digit character.
<code>\b</code>	Batas kata.

Tabel 14: Komponen kunci dalam materi String Matching with Regex

8.6 Algoritma dan Rumus Penting

8.6.1 Penggunaan Dasar Regex di Python

Tujuan Algoritma.

Contoh ini memperlihatkan alur dasar membuat pattern dan mencari semua kecocokan yang tidak saling tumpang tindih.

Listing 6: Penggunaan dasar regex di Python

```
1 import re
2
3 pattern = re.compile(r"\d{4}")
4 hasil = pattern.findall("Tahun 2024 dan 2026")
5 print(hasil)
```

8.7 Checklist Pemahaman

8.7.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan regex sebagai notasi pattern.
- ☐ Saya dapat membedakan exact matching dan regex matching.
- ☐ Saya dapat menggunakan character class, range, negasi, dan quantifier.
- ☐ Saya dapat menjelaskan fungsi `re.compile`, `search`, `match`, dan `findall`.
- ☐ Saya dapat merancang regex sederhana dari spesifikasi format teks.

8.7.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Character class, range, negasi, quantifier, boundary matcher, dan fungsi dasar Python regex.
Cukup Pahami Konsep	Contoh email, nomor telepon, dan ELIZA-like chatbot.
Sudah dikuasai	Belum ditentukan.

9 Program Dinamis

Program Dinamis (*dynamic programming*) adalah strategi perancangan algoritma untuk persoalan optimisasi yang dapat dipecah menjadi subpersoalan bertahap. Inti metodenya adalah mendefinisikan status, menentukan keputusan pada setiap tahap, menyusun relasi rekurens, menyimpan nilai subpersoalan dalam tabel, lalu merekonstruksi solusi optimal jika diperlukan.

Fokus UAS adalah kemampuan memformulasikan persoalan, bukan sekadar mengisi angka. Jawaban Program Dinamis yang baik selalu menjelaskan tahap, status, keputusan, basis, rekurens, tabel perhitungan, dan rekonstruksi solusi. Pola ini terlihat pada soal pencuri rumah, shortest path bertahap, pemilihan station, dan jalur terpendek dengan DP maju/mundur.

Komentar 9.1: Keterbatasan Sumber Utama

NotebookLM yang diberikan dapat diakses, tetapi jawaban yang diperoleh untuk Program Dinamis hanya memuat outline global: tahap, status, prinsip optimalitas, memoization, pendekatan maju/mundur, serta aplikasi shortest path, integer knapsack, coin-row, capital budgeting, dan TSP. Detail teknis seperti pola tabel UAS, rekonstruksi solusi, dan edge case dilengkapi menggunakan pengetahuan umum yang relevan serta pola soal UAS 2022–2025.

9.1 Outline Fundamental Konsep Materi

1. Definisi dan motivasi Program Dinamis
 - (a) Persoalan optimisasi.
 - (b) Rangkaian keputusan.

- (c) Subpersoalan yang saling tumpang tindih.
 - (d) Penyimpanan nilai subpersoalan.
2. Karakteristik persoalan DP
- (a) Tahap (*stage*).
 - (b) Status (*state*).
 - (c) Keputusan (*decision*).
 - (d) Transisi status.
 - (e) Fungsi nilai optimal.
 - (f) Prinsip optimalitas.
3. Cara membangun solusi DP
- (a) Menentukan struktur solusi optimal.
 - (b) Mendefinisikan fungsi nilai.
 - (c) Menentukan basis.
 - (d) Menentukan relasi rekurens.
 - (e) Mengisi tabel secara maju atau mundur.
 - (f) Merekonstruksi solusi.
4. Pendekatan perhitungan
- (a) Top-down dengan memoization.
 - (b) Bottom-up dengan tabulation.
 - (c) Program dinamis maju.
 - (d) Program dinamis mundur.
5. Aplikasi penting
- (a) Shortest path bertahap.
 - (b) House robber atau coin-row.
 - (c) Integer knapsack.
 - (d) Coin exchange.
 - (e) Capital budgeting.
 - (f) Travelling Salesperson Problem (TSP) dengan DP.

9.2 Pentingnya Materi dan Relevansinya

Program Dinamis penting karena banyak persoalan optimisasi tidak dapat diselesaikan dengan keputusan lokal yang sederhana. Greedy memilih keputusan terbaik saat ini, tetapi tidak selalu menghasilkan solusi global optimal. DP menghindari jebakan itu dengan menyimpan hasil optimal subpersoalan dan memastikan setiap keputusan dilihat dalam konteks tahap/status yang tepat.

Dalam UAS, Program Dinamis biasanya menilai kedisiplinan formulasi. Jika tahap, status, dan rekurens salah, tabel angka yang dihasilkan hampir pasti salah. Sebaliknya, jika formulasi benar, pengisian tabel biasanya menjadi proses mekanis: setiap sel dihitung dari sel yang sudah tersedia.

Komentar 9.2: Relevansi Praktis

Program Dinamis banyak muncul pada optimisasi rute, alokasi sumber daya, perencanaan produksi, bioinformatika, kompresi, dan penjadwalan. Catatan ini berfokus pada bentuk yang paling relevan untuk UAS: shortest path, knapsack, coin exchange, capital budgeting, TSP, dan persoalan pemilihan berurutan seperti pencuri rumah.

9.2.1 Dependensi dan Hubungan dengan Materi Lain

Program Dinamis berhubungan dengan Route Planning melalui persoalan lintasan terpendek. Route Planning mengekspansi simpul dari frontier, sedangkan DP memecah graf ke dalam tahap dan menyimpan nilai minimum per status. Keduanya dapat menghasilkan jalur terpendek, tetapi bentuk jawaban dan struktur perhitungannya berbeda.

DP juga berhubungan dengan Branch and Bound pada persoalan optimisasi seperti knapsack dan TSP. Branch and Bound membangun pohon ruang status dan memangkas simpul berdasarkan bound, sedangkan DP membangun tabel subpersoalan agar kombinasi keputusan tidak dihitung berulang. Hubungan dengan Teori P/NP/NPC muncul karena beberapa persoalan seperti TSP dan knapsack memiliki versi yang sulit; DP dapat menyelesaikan varian tertentu dengan struktur atau batas ukuran tertentu, tetapi tidak otomatis membuat semua versi menjadi mudah.

9.3 Pola Soal UAS Sebelumnya dan Prioritas

Komentar 9.3: Pola dari Soal 2022–2025

Program Dinamis muncul pada semua sumber soal yang diperiksa. Soal 2022 memakai persoalan pencuri rumah: brute force, rekurens, dan perhitungan contoh. Solusi 2023 memakai DP backward pada pemilihan station. Soal 2024 meminta rute terpendek dengan tabel per tahap dan rekonstruksi solusi. Soal 2025 meminta shortest path pada graf yang sama dengan route planning, tetapi harus dikerjakan dengan DP maju dan tabel dinamis.

Pola Soal	Yang Biasanya Diminta	Kemampuan Wajib
Formulasi rekurens	Basis dan relasi rekursif untuk fungsi nilai optimal.	Menentukan status, keputusan, dan kasus dasar secara eksplisit.
Tabel DP per tahap	Mengisi tabel maju atau mundur.	Menghitung setiap sel dari sel tahap sebelumnya/berikutnya, bukan menebak hasil akhir.
Rekonstruksi solusi	Menuliskan jalur, pilihan rumah, item, atau station yang membentuk solusi optimal.	Menyimpan keputusan optimal atau melakukan traceback dari tabel.
Perbandingan dengan brute force	Menjelaskan nilai maksimum/minimum dan mengapa DP menghindari enumerasi penuh.	Memahami overlapping subproblems dan prinsip optimalitas.

Tabel 15: Pola soal UAS Program Dinamis

9.4 Penjelasan Konsep Fundamental

9.4.1 Definisi Program Dinamis

Inti Konsep.

Program Dinamis menyelesaikan persoalan optimisasi dengan menyusun solusi sebagai rangkaian keputusan dan menyimpan nilai optimal subpersoalan.

Kata “program” pada Program Dinamis berarti perencanaan, bukan kode program. Suatu persoalan cocok untuk DP jika solusi optimalnya dapat dibangun dari solusi optimal subpersoalan

yang lebih kecil. Jika subpersoalan yang sama muncul berulang kali, tabel DP menghindari penghitungan ulang.

DP biasanya dipakai untuk persoalan maksimum atau minimum. Contohnya: biaya minimum mencapai tujuan, profit maksimum dengan kapasitas tertentu, jumlah koin minimum untuk nilai tertentu, atau nilai maksimum dari pilihan yang tidak boleh bertetangga.

9.4.2 Tahap, Status, Keputusan, dan Transisi

Inti Konsep.

Tahap menyatakan urutan pengambilan keputusan, status menyatakan informasi yang masih relevan pada tahap itu, dan keputusan mengubah satu status ke status lain.

Empat komponen ini harus ditulis jelas dalam jawaban:

- **Tahap:** indeks waktu, posisi, item, rumah, atau lapisan graf yang sedang diproses.
- **Status:** informasi minimum yang diperlukan untuk melanjutkan keputusan secara optimal.
- **Keputusan:** pilihan yang tersedia pada suatu tahap/status.
- **Transisi:** perubahan status akibat keputusan, termasuk biaya atau profit yang ditambahkan.

Status harus cukup informatif tetapi tidak berlebihan. Jika status terlalu miskin, rekurens tidak valid karena informasi penting hilang. Jika status terlalu kaya, tabel menjadi besar dan sulit dihitung.

9.4.3 Prinsip Optimalitas

Inti Konsep.

Prinsip optimalitas menyatakan bahwa bagian dari solusi optimal juga harus optimal untuk subpersoalan yang sesuai.

Misalnya jalur terpendek dari A ke F melewati D . Maka bagian jalur dari A ke D harus merupakan jalur terpendek untuk mencapai D dalam struktur tahap yang sama. Jika tidak, bagian tersebut dapat diganti dengan jalur yang lebih murah dan menghasilkan jalur total yang lebih baik, sehingga jalur awal bukan optimal.

Prinsip ini menjadi alasan rekurens DP benar. Tanpa prinsip optimalitas, menyimpan nilai optimal subpersoalan dapat menyesatkan karena pilihan terbaik lokal untuk subpersoalan belum tentu kompatibel dengan solusi global.

9.4.4 Memoization dan Tabulation

Inti Konsep.

Memoization menyimpan hasil fungsi rekursif saat dibutuhkan, sedangkan tabulation mengisi tabel secara sistematis dari basis menuju solusi akhir.

Top-down memoization dimulai dari persoalan utama, memanggil subpersoalan secara rekursif, lalu menyimpan hasilnya. Bottom-up tabulation dimulai dari subpersoalan basis, mengisi tabel sampai mencapai persoalan utama. Untuk UAS tertulis, bentuk tabulation lebih sering diminta karena mahasiswa harus menampilkan tabel per tahap.

9.4.5 Program Dinamis Maju dan Mundur

Inti Konsep.

DP maju menghitung nilai dari tahap awal ke tahap akhir, sedangkan DP mundur menghitung nilai dari tahap akhir ke tahap awal.

Pada DP maju, nilai suatu status biasanya menyatakan biaya/profit terbaik untuk mencapai status tersebut dari awal. Pada DP mundur, nilai suatu status menyatakan biaya/profit terbaik dari status tersebut menuju akhir. Keduanya dapat menghasilkan solusi yang sama jika rekurens dan basisnya benar.

Dalam soal shortest path bertahap, DP maju cocok jika graf disusun dari sumber ke tujuan. DP mundur cocok jika lebih mudah menghitung biaya minimum dari setiap simpul menuju tujuan.

9.5 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Tahap	Urutan keputusan atau lapisan subpersoalan.
Status	Informasi yang membedakan subpersoalan pada tahap tertentu.
Keputusan	Pilihan yang tersedia pada status tertentu.
Transisi	Perubahan status akibat keputusan.
Fungsi nilai f	Nilai optimum suatu subpersoalan, misalnya biaya minimum atau profit maksimum.
Basis	Nilai awal rekurens yang diketahui langsung.
Relasi rekurens	Persamaan yang menghubungkan nilai suatu status dengan subpersoalan yang lebih kecil.
Tabel DP	Penyimpanan nilai subpersoalan.
Traceback	Proses merekonstruksi solusi dari keputusan optimal yang tersimpan.

Tabel 16: Komponen kunci dalam materi Program Dinamis

9.6 Algoritma dan Rumus Penting

9.6.1 Skema Umum Program Dinamis

Tujuan Algoritma.

Skema ini membantu mengubah persoalan optimisasi menjadi tabel DP yang dapat dihitung.

Listing 7: Langkah pengembangan Program Dinamis

```
1 1. Tentukan tahap, status, dan keputusan.
2 2. Definisikan fungsi nilai optimal  $f(\dots)$ .
3 3. Tentukan basis atau kondisi awal.
4 4. Turunkan relasi rekurens dari pilihan keputusan.
5 5. Tentukan arah pengisian tabel: maju atau mundur.
6 6. Isi tabel sampai nilai optimum persoalan utama diperoleh.
7 7. Rekonstruksi solusi dari keputusan optimal.
```

Skema umum minimisasi:

$$f_k(s) = \min_{x \in D_k(s)} \{c_k(s, x) + f_{k+1}(T_k(s, x))\} \quad (37)$$

dengan:

- k : tahap.
- s : status pada tahap k .
- x : keputusan.
- $D_k(s)$: himpunan keputusan yang valid.
- $c_k(s, x)$: biaya keputusan.
- $T_k(s, x)$: status baru setelah keputusan x .

Untuk maksimisasi, operator min diganti menjadi max dan biaya dapat diganti profit.

9.6.2 Shortest Path Bertahap

Makna Rumus.

Rumus shortest path DP menghitung biaya minimum mencapai suatu simpul dari simpul-simpul pendahulu pada tahap sebelumnya.

DP maju:

$$f_k(v) = \min_{u \in \text{Pred}(v)} \{f_{k-1}(u) + c(u, v)\} \quad (38)$$

DP mundur:

$$F_k(u) = \min_{v \in \text{Succ}(u)} \{c(u, v) + F_{k+1}(v)\} \quad (39)$$

dengan:

- $\text{Pred}(v)$: simpul pendahulu v .
- $\text{Succ}(u)$: simpul penerus u .
- $c(u, v)$: biaya sisi dari u ke v .

Untuk UAS, tabel shortest path harus disertai rekonstruksi jalur. Simpan simpul pendahulu yang memberi nilai minimum pada DP maju, atau simpul penerus yang memberi nilai minimum pada DP mundur.

9.6.3 House Robber atau Coin-Row

Makna Rumus.

Rumus ini memilih antara melewati item ke- i atau mengambil item ke- i sehingga item sebelumnya tidak boleh diambil.

$$F(0) = 0, \quad (40)$$

$$F(1) = p_1, \quad (41)$$

$$F(i) = \max\{F(i-1), F(i-2) + p_i\}. \quad (42)$$

dengan:

- p_i : nilai rumah atau koin ke- i .
- $F(i-1)$: nilai maksimum jika item ke- i tidak diambil.
- $F(i-2) + p_i$: nilai maksimum jika item ke- i diambil.

Rekonstruksi dilakukan dari belakang: jika $F(i) = F(i-1)$, item ke- i tidak diambil; jika $F(i) = F(i-2) + p_i$, item ke- i diambil dan lanjut ke $i-2$.

9.6.4 Integer Knapsack

Makna Rumus.

Rumus knapsack memilih nilai maksimum antara tidak mengambil objek ke- k atau mengambilnya jika kapasitas cukup.

$$f_k(y) = \begin{cases} f_{k-1}(y), & y < w_k, \\ \max\{f_{k-1}(y), p_k + f_{k-1}(y - w_k)\}, & y \geq w_k. \end{cases} \quad (43)$$

dengan:

- k : jumlah item pertama yang dipertimbangkan.
- y : kapasitas yang tersedia.
- w_k : bobot item ke- k .
- p_k : profit item ke- k .

Untuk 0/1 knapsack, setiap item hanya boleh diambil sekali. Karena itu saat item ke- k diambil, nilai sebelumnya berasal dari $f_{k-1}(y - w_k)$, bukan dari $f_k(y - w_k)$.

9.6.5 Coin Exchange

Makna Rumus.

Coin exchange biasanya mencari jumlah koin minimum untuk membentuk nilai tertentu.

Jika tersedia denominasi koin $d_1, d_2, \dots, d_r$, rumus minimum jumlah koin:

$$F(x) = 1 + \min_{d_i \leq x} F(x - d_i) \quad (44)$$

dengan basis:

$$F(0) = 0 \quad (45)$$

Jika suatu nilai tidak dapat dibentuk, nilainya dianggap tak hingga agar tidak dipilih dalam operasi minimum.

9.6.6 TSP dengan Program Dinamis

Makna Rumus.

DP untuk TSP menyimpan biaya minimum mencapai simpul terakhir tertentu setelah mengunjungi himpunan kota tertentu.

Salah satu formulasi umum adalah Held-Karp:

$$C(S, j) = \min_{i \in S, i \neq j} \{C(S - \{j\}, i) + c(i, j)\} \quad (46)$$

dengan:

- S : himpunan kota yang sudah dikunjungi.
- j : kota terakhir dalam lintasan.
- $c(i, j)$: biaya dari kota i ke kota j .

Formulasi ini menunjukkan bahwa DP dapat mengurangi enumerasi penuh, tetapi tetap mahal untuk jumlah kota besar karena statusnya berupa himpunan.

9.7 Cara Menjawab Soal Manual

9.7.1 Template Jawaban Formulasi DP

1. **Tahap:** tuliskan apa yang menjadi urutan tahap.
2. **Status:** tuliskan arti indeks atau parameter fungsi nilai.
3. **Keputusan:** tuliskan pilihan yang tersedia.
4. **Fungsi nilai:** definisikan $f(k, s)$ atau notasi lain.
5. **Basis:** tuliskan nilai awal.
6. **Rekurens:** tuliskan persamaan minimum/maksimum.
7. **Tabel:** isi nilai per tahap.
8. **Rekonstruksi:** tuliskan solusi konkret, bukan hanya nilai optimum.

9.7.2 Template Tabel Shortest Path

Tahap	Status	Perhitungan	Keputusan Optimal
k	v	$f_k(v) = \min\{\dots\}$	Pendahulu/penerus terbaik

Tabel 17: Template tabel DP untuk shortest path

9.8 Kesalahan Umum dan Edge Case

- **Tidak mendefinisikan status:** tabel tanpa definisi status sulit dinilai karena pembaca tidak tahu arti sel.
- **Basis hilang:** rekurens tidak bisa dievaluasi jika nilai awal tidak jelas.
- **Mencampur DP maju dan mundur:** arah rekurens harus konsisten dengan tabel.
- **Tidak merekonstruksi solusi:** banyak soal meminta jalur atau pilihan optimal, bukan hanya nilai.
- **Menggunakan greedy sebagai DP:** memilih nilai terbesar saat ini bukan DP kecuali dibuktikan cocok dengan rekurens.
- **Status tidak cukup:** jika status tidak menyimpan informasi penting, keputusan berikutnya dapat salah.
- **Knapsack salah indeks:** 0/1 knapsack memakai baris $k - 1$ saat mengambil item, bukan baris k .
- **Tie tidak dijelaskan:** jika ada dua keputusan sama optimal, tuliskan salah satu dan sebutkan ada alternatif.

9.9 Checklist Pemahaman

9.9.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan tujuan Program Dinamis dan perbedaannya dari Greedy.
- ☐ Saya dapat menentukan tahap, status, keputusan, transisi, basis, dan fungsi nilai.
- ☐ Saya dapat menjelaskan prinsip optimalitas dan overlapping subproblems.
- ☐ Saya dapat membedakan memoization dan tabulation.
- ☐ Saya dapat membedakan DP maju dan DP mundur.
- ☐ Saya dapat menyusun rekurens shortest path bertahap.
- ☐ Saya dapat menyusun rekurens house robber/coin-row.
- ☐ Saya dapat menyusun rekurens integer knapsack.
- ☐ Saya dapat mengisi tabel DP dan merekonstruksi solusi optimal.
- ☐ Saya dapat menjelaskan hubungan DP dengan Route Planning, Branch and Bound, dan Teori P/NP/NPC.

9.9.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib Dikuasai	Tahap, status, keputusan, basis, relasi rekurens, tabel DP, rekonstruksi solusi, shortest path bertahap, dan house robber/coin-row.
Cukup Paham Konsep	Integer knapsack, coin exchange, capital budgeting, TSP dengan DP, memoization versus tabulation, dan DP maju versus mundur.
Sudah Dikuasai	Belum ditentukan.

10 Teori P, NP, NPC

Teori P, NP, dan NP-Complete membahas klasifikasi persoalan keputusan berdasarkan efisiensi penyelesaian, efisiensi verifikasi, dan hubungan reduksi antar persoalan. Materi ini menjawab pertanyaan: apakah suatu persoalan dapat diselesaikan dalam waktu polinomial, apakah solusi yang diberikan dapat diverifikasi dalam waktu polinomial, dan apakah persoalan tersebut sesukar seluruh persoalan di NP.

Fokus UAS biasanya berbentuk benar/salah atau pilihan ganda konseptual. Karena itu, yang paling penting adalah definisi yang presisi: P bukan sekadar “mudah”, NP bukan “tidak polinomial”, NP-Complete harus berada di NP sekaligus NP-Hard, dan NP-Hard tidak harus berada di NP.

Komentar 10.1: Keterbatasan Sumber Utama

NotebookLM yang diberikan dapat diakses, tetapi jawaban yang diperoleh untuk Teori P, NP, NPC hanya memuat outline global: polynomial-time, tractable, undecidable, deterministic/non-deterministic algorithm, P, NP, NP-Complete, NP-Hard, reduksi polinomial, SAT, TSP decision problem, dan hubungan dengan materi lain. Detail klasifikasi contoh, jebakan benar/salah, dan konsekuensi diagram Venn dilengkapi menggunakan pengetahuan umum yang relevan serta pola soal UAS 2022–2025.

10.1 Outline Fundamental Konsep Materi

1. Persoalan dan kompleksitas
 - (a) Persoalan keputusan.
 - (b) Persoalan optimisasi.
 - (c) Ukuran masukan.
 - (d) Kompleksitas waktu.
 - (e) Polynomial-time dan nonpolynomial-time.
2. Keterpecahan persoalan
 - (a) Tractable.
 - (b) Intractable.
 - (c) Decidable.
 - (d) Undecidable.
 - (e) Halting problem.
3. Model algoritma
 - (a) Algoritma deterministik.
 - (b) Algoritma non-deterministik.
 - (c) Tahap menerka.
 - (d) Tahap verifikasi.
 - (e) Sertifikat solusi.
4. Kelas kompleksitas
 - (a) P .
 - (b) NP .
 - (c) NP-Complete (NPC).
 - (d) NP-Hard.
 - (e) Relasi $P \subseteq NP$.
5. Reduksi dan pembuktian NPC
 - (a) Reduksi polinomial.
 - (b) $A \leq_p B$.
 - (c) Syarat $X \in NP$.
 - (d) Syarat semua persoalan NP dapat direduksi ke X .
 - (e) SAT sebagai persoalan NPC dasar.
6. Contoh penting untuk ujian
 - (a) SAT.
 - (b) Hamiltonian Circuit.
 - (c) TSP decision problem.
 - (d) Clique decision problem.
 - (e) Euler circuit.
 - (f) Minimum spanning tree.
 - (g) Halting problem.

10.2 Pentingnya Materi dan Relevansinya

Materi ini penting karena strategi algoritma tidak hanya membahas cara menyelesaikan persoalan, tetapi juga batas realistis penyelesaiannya. Jika sebuah persoalan berada di P , kita dapat mencari algoritma polinomial yang eksak. Jika sebuah persoalan NP-Complete, kita perlu sadar bahwa algoritma polinomial eksak untuk semua instance belum diketahui; pendekatan seperti backtracking, Branch and Bound, heuristik, aproksimasi, atau DP pada varian terbatas menjadi relevan.

Komentar 10.2: Relevansi Praktis

Dalam praktik, klasifikasi P /NP/NPC membantu memilih pendekatan: algoritma eksak polinomial untuk persoalan tractable, optimisasi dengan pruning untuk instance kecil, heuristik atau aproksimasi untuk instance besar, dan reformulasi persoalan jika model awal terlalu sulit.

10.2.1 Dependensi dan Hubungan dengan Materi Lain

Backtracking, Branch and Bound, dan Program Dinamis sering dipakai pada persoalan yang ruang kemungkinannya besar. Teori P /NP/NPC menjelaskan mengapa strategi seperti pruning, bound, heuristic search, atau tabel DP dibutuhkan. Contohnya, Hamiltonian Circuit dan TSP decision problem terkait dengan NPC, sedangkan TSP optimisasi terkait NP-Hard. Knapsack juga memiliki varian yang sering dibahas dalam konteks NP-Complete/NP-Hard, walaupun DP dapat menyelesaikan versi tertentu dalam waktu pseudo-polinomial.

10.3 Pola Soal UAS Sebelumnya dan Prioritas

Komentar 10.3: Pola dari Soal 2022–2025

Teori P , NP, dan NPC muncul pada semua sumber soal yang diperiksa. Soal 2022 berbentuk pilihan ganda tentang tractable, algoritma non-deterministik, tahap pembuktian NPC, contoh bukan NPC, asumsi $P \neq NP$, dan diagram Venn. Solusi 2023 berisi benar/salah untuk Menara Hanoi, FFT, Halting Problem, dan Post Correspondence Problem. Soal 2024 berisi benar/salah tentang eigen, MST, SAT, P subset NP, dan P versus NP. Soal 2025 berisi pilihan ganda tentang Euler circuit, contoh NP, tahap pembuktian NPC, dampak algoritma polinomial untuk persoalan NPC, Halting Problem, dan diagram Venn.

Pola Soal	Yang Biasanya Diminta	Kemampuan Wajib
Klasifikasi persoalan	Menentukan apakah persoalan termasuk P , NP, NPC, NP-Hard, intractable, atau undecidable.	Mengingat definisi dan contoh kanonik.
Benar/salah definisi	Menilai pernyataan tentang P , NP, verifikasi, deterministik, dan non-deterministik.	Tidak tertukar antara menyelesaikan dan memverifikasi.
Pembuktian NPC	Menjawab tahap/syarat membuktikan suatu persoalan NP-Complete.	Menyebut $X \in NP$ dan reduksi dari persoalan NPC/NP yang diketahui.
Konsekuensi $P = NP$	Menentukan implikasi jika satu persoalan NPC memiliki algoritma polinomial.	Memahami bahwa satu NPC di P membuat semua NP berada di P .
Diagram Venn	Memilih diagram hubungan P , NP, dan NPC.	Memahami $P \subseteq NP$, NPC subset NP, dan skenario jika $P = NP$.

Tabel 18: Pola soal UAS Teori P , NP, NPC

10.4 Penjelasan Konsep Fundamental

10.4.1 Persoalan Keputusan dan Persoalan Optimisasi

Inti Konsep.

Kelas P, NP, dan NPC secara formal membahas persoalan keputusan, yaitu persoalan dengan jawaban ya atau tidak.

Persoalan optimisasi mencari nilai terbaik, misalnya jarak terpendek atau profit maksimum. Persoalan keputusan hanya menanyakan apakah ada solusi yang memenuhi batas tertentu. Contoh TSP optimisasi: “berapakah tur terpendek?” Contoh TSP decision problem: “apakah ada tur dengan panjang paling banyak K ?” Versi keputusan inilah yang dibahas dalam kelas NP dan NP-Complete.

Perbedaan ini sering menjadi jebakan. TSP optimisasi biasanya dikaitkan dengan NP-Hard, sedangkan TSP decision problem berada di NP-Complete.

10.4.2 Polynomial-time, Tractable, dan Intractable

Inti Konsep.

Polynomial-time berarti waktu algoritma dibatasi oleh polinom terhadap ukuran masukan; persoalan yang memiliki algoritma polinomial dianggap tractable.

Contoh waktu polinomial adalah $O(1)$, $O(n)$, $O(n \log n)$, $O(n^2)$, dan $O(n^3)$. Contoh nonpolinomial adalah $O(2^n)$, $O(n!)$, dan sejenisnya. Secara praktis, polynomial-time dipakai sebagai batas antara persoalan yang dianggap dapat ditangani efisien dan yang tidak.

Intractable berarti tidak dapat diselesaikan dalam waktu polinomial, atau sejauh diketahui tidak memiliki algoritma polinomial untuk penyelesaian eksak umum. Namun, intractable berbeda dari undecidable. Intractable masih dapat diputuskan oleh algoritma, hanya saja membutuhkan waktu yang sangat besar untuk instance besar.

10.4.3 Decidable dan Undecidable

Inti Konsep.

Decidable berarti ada algoritma yang selalu berhenti dan memberi jawaban benar; undecidable berarti tidak ada algoritma umum yang selalu memutuskan jawaban.

Halting Problem adalah contoh undecidable: diberikan program dan input, tentukan apakah program tersebut akan berhenti. Tidak ada algoritma umum yang selalu dapat menjawab benar untuk semua program-input. Karena undecidable tidak memiliki algoritma pemutus umum, ia tidak sekadar “sangat lambat”; persoalannya berbeda dari intractable.

10.4.4 Algoritma Deterministik dan Non-deterministik

Inti Konsep.

Algoritma deterministik memiliki langkah berikutnya yang pasti, sedangkan model non-deterministik dapat dipahami sebagai menebak sertifikat solusi lalu memverifikasinya.

Dalam materi NP, non-deterministik tidak berarti acak. Model yang penting untuk ujian adalah dua tahap:

1. **Guessing:** menerka sertifikat solusi.
2. **Verification:** memeriksa sertifikat secara deterministik dalam waktu polinomial.

Sebuah persoalan berada di NP jika setiap jawaban “ya” memiliki sertifikat yang dapat diverifikasi dalam waktu polinomial. Ini tidak berarti solusi dapat ditemukan dalam waktu polinomial oleh algoritma deterministik.

10.4.5 Kelas P

Inti Konsep.

P adalah kelas persoalan keputusan yang dapat diselesaikan oleh algoritma deterministik dalam waktu polinomial.

Jika suatu persoalan berada di P, maka ada algoritma yang dapat menentukan jawaban ya/tidak secara efisien. Contoh yang sering muncul di pola soal adalah Minimum Spanning Tree dan Euler Circuit. Keduanya memiliki algoritma polinomial, sehingga versi keputusannya berada di P. Karena setiap solusi yang dapat diselesaikan cepat juga dapat diverifikasi cepat, setiap persoalan P juga berada di NP.

10.4.6 Kelas NP

Inti Konsep.

NP adalah kelas persoalan keputusan yang solusi “ya”-nya dapat diverifikasi dalam waktu polinomial.

Contohnya Hamiltonian Circuit: jika seseorang memberi urutan simpul sebagai sertifikat, kita dapat memeriksa dalam waktu polinomial apakah urutan itu melewati setiap simpul tepat sekali dan kembali ke awal. TSP decision problem juga berada di NP: sertifikatnya adalah tur, dan verifikasi memeriksa validitas tur serta apakah total bobot tidak melebihi batas.

Pernyataan “NP berarti non-polynomial” adalah salah. NP berarti *nondeterministic polynomial*, yaitu terkait verifikasi polinomial dalam model non-deterministik.

10.4.7 NP-Complete

Inti Konsep.

NP-Complete adalah persoalan di NP yang setidaknya sesukar semua persoalan lain di NP.

Sebuah persoalan X termasuk NP-Complete jika memenuhi dua syarat:

1. $X \in NP$.
2. Untuk setiap persoalan $Y \in NP$, berlaku $Y \leq_p X$.

Dalam praktik pembuktian, kita tidak mereduksi semua Y satu per satu. Kita mengambil persoalan yang sudah diketahui NP-Complete, misalnya SAT atau Hamiltonian Circuit, lalu mereduksinya ke persoalan X dalam waktu polinomial. Jika A sudah NP-Complete dan $A \leq_p X$, maka X setidaknya sesukar A .

10.4.8 NP-Hard

Inti Konsep.

NP-Hard adalah kelas persoalan yang setidaknya sesukar semua persoalan NP, tetapi tidak harus berada di NP.

Setiap persoalan NP-Complete adalah NP-Hard, tetapi tidak semua NP-Hard adalah NP-Complete. Persoalan NP-Hard dapat berupa persoalan optimisasi, bukan keputusan, atau

bahkan tidak decidable. Karena itu, untuk menyebut suatu persoalan NP-Complete, kita harus memastikan persoalan tersebut berada di NP. Untuk NP-Hard, syarat berada di NP tidak diperlukan.

10.5 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Persoalan keputusan	Persoalan dengan jawaban ya/tidak; objek formal utama kelas P, NP, dan NPC.
Polynomial-time	Waktu algoritma dibatasi fungsi polinom ukuran input.
Tractable	Dapat diselesaikan dalam waktu polinomial.
Intractable	Tidak dapat diselesaikan dalam waktu polinomial untuk penyelesaian eksak umum.
Undecidable	Tidak ada algoritma umum yang selalu berhenti dan menjawab benar.
P	Persoalan keputusan yang dapat diselesaikan dalam waktu polinomial.
NP	Persoalan keputusan yang sertifikat solusi “ya”-nya dapat diverifikasi dalam waktu polinomial.
NPC	Persoalan yang berada di NP dan semua persoalan NP dapat direduksi kepadanya.
NP-Hard	Persoalan yang semua persoalan NP dapat direduksi kepadanya, tidak harus berada di NP.
Reduksi polinomial	Transformasi instance persoalan satu ke persoalan lain dalam waktu polinomial.
Sertifikat	Bukti kandidat jawaban “ya” yang dapat diverifikasi.

Tabel 19: Komponen kunci dalam materi Teori P, NP, NPC

10.6 Algoritma dan Rumus Penting

10.6.1 Kelas P

Makna Rumus.

Definisi ini menyatakan bahwa P berisi persoalan keputusan yang dapat diputuskan secara deterministik dalam waktu polinomial.

$$P = \{L \mid L \text{ dapat diputuskan oleh algoritma deterministik dalam waktu polinomial}\} \quad (47)$$

10.6.2 Kelas NP

Makna Rumus.

Definisi ini menyatakan bahwa NP berisi persoalan yang jawaban “ya”-nya memiliki sertifikat yang dapat diverifikasi cepat.

$$NP = \{L \mid \exists V \text{ polinomial, } x \in L \iff \exists y, V(x, y) = 1\} \quad (48)$$

dengan:

- x : instance persoalan.

- y : sertifikat solusi.
- V : verifier deterministik waktu polinomial.

10.6.3 Relasi P dan NP

Makna Rumus.

Jika suatu persoalan dapat diselesaikan cepat, maka solusi yang diberikan juga dapat diverifikasi cepat.

$$P \subseteq NP \quad (49)$$

Pertanyaan besar yang belum diketahui adalah apakah:

$$P = NP \quad \text{atau} \quad P \neq NP \quad (50)$$

10.6.4 Reduksi Polinomial

Makna Rumus.

Reduksi polinomial menunjukkan bahwa jika B mudah diselesaikan, maka A juga mudah diselesaikan melalui transformasi ke B .

$$A \leq_p B \iff \exists f \text{ komputabel polinomial sehingga } x \in A \iff f(x) \in B \quad (51)$$

Makna arah reduksi sangat penting. Jika $A \leq_p B$, maka B setidaknya sesukar A . Untuk membuktikan B sulit, reduksi dilakukan dari persoalan sulit yang sudah dikenal menuju B .

10.6.5 Syarat NP-Complete dan NP-Hard

Makna Rumus.

NP-Complete membutuhkan dua syarat: berada di NP dan NP-Hard.

$$X \in NPC \iff X \in NP \wedge \forall Y \in NP, Y \leq_p X \quad (52)$$

$$X \in NP\text{-Hard} \iff \forall Y \in NP, Y \leq_p X \quad (53)$$

Konsekuensi penting:

$$\exists X \in NPC, X \in P \Rightarrow P = NP \quad (54)$$

Artinya, jika satu saja persoalan NP-Complete ditemukan memiliki algoritma polinomial, maka seluruh persoalan di NP juga dapat diselesaikan dalam polinomial.

10.7 Contoh Klasifikasi yang Sering Muncul

Persoalan	Klasifikasi Umum	Catatan Ujian
Minimum Spanning Tree	P	Ada algoritma polinomial seperti Kruskal dan Prim.
Euler Circuit	P	Dapat diperiksa dengan derajat simpul dan keterhubungan dalam waktu polinomial.
Hamiltonian Circuit	NP-Complete	Sertifikatnya urutan simpul; verifikasi polinomial.
SAT	NP-Complete	Contoh dasar NPC yang sering menjadi titik awal reduksi.
TSP decision problem	NP-Complete	Pertanyaan ya/tidak: apakah ada tur dengan biaya paling banyak K .
TSP optimisasi	NP-Hard	Mencari tur minimum, bukan sekadar menjawab ya/tidak.
Clique decision problem	NP-Complete	Jika versi decision clique polinomial, maka $P = NP$.
Halting Problem	Undecidable	Tidak termasuk sekadar NPC; tidak ada algoritma umum yang selalu memutuskan.
GCD dua bilangan	P	Dapat diselesaikan dengan algoritma Euclid dalam waktu polinomial.

Tabel 20: Contoh klasifikasi yang sering relevan pada soal

10.8 Jebakan Benar/Salah yang Perlu Diwaspadai

- **NP bukan non-polynomial:** NP berarti solusi dapat diverifikasi dalam waktu polinomial oleh model non-deterministik.
- **P adalah subset NP:** karena jika persoalan dapat diselesaikan cepat, hasilnya juga dapat diverifikasi cepat.
- **NPC tidak sama dengan NP:** NPC adalah bagian dari NP yang paling sulit.
- **NP-Hard tidak harus berada di NP:** karena bisa berupa persoalan optimisasi atau bahkan undecidable.
- **Undecidable berbeda dari intractable:** intractable masih decidable tetapi tidak efisien; undecidable tidak memiliki algoritma pemutus umum.
- **Satu NPC di P membuat $P = NP$:** bukan hanya persoalan itu yang menjadi mudah, tetapi seluruh NP.
- **Jika $P \neq NP$:** maka tidak ada persoalan NP-Complete yang berada di P.
- **Verifikasi bukan pencarian:** mampu memeriksa sertifikat cepat tidak berarti mampu menemukan sertifikat cepat.

10.9 Cara Menjawab Soal Manual

10.9.1 Menentukan Apakah Persoalan Berada di NP

1. Pastikan persoalan adalah persoalan keputusan.
2. Tentukan bentuk sertifikat untuk jawaban “ya”.
3. Jelaskan verifier deterministik.
4. Pastikan verifier berjalan dalam waktu polinomial terhadap ukuran input.

10.9.2 Membuktikan NP-Complete

1. Buktikan $X \in NP$ dengan sertifikat dan verifier polinomial.
2. Pilih persoalan A yang sudah diketahui NP-Complete.
3. Tunjukkan reduksi polinomial $A \leq_p X$.
4. Simpulkan X NP-Hard dan karena $X \in NP$, maka X NP-Complete.

10.9.3 Menganalisis Konsekuensi Algoritma Polinomial

Jika soal menyatakan ada algoritma polinomial untuk persoalan NP-Complete seperti SAT, Hamiltonian Circuit, Clique, atau TSP decision problem, maka konsekuensinya:

- persoalan tersebut berada di P;
- semua persoalan NP dapat direduksi ke persoalan tersebut;
- semua persoalan NP juga berada di P;
- maka $P = NP$.

10.10 Checklist Pemahaman

10.10.1 Submateri yang Telah Dibahas

- ☐ Saya dapat membedakan persoalan keputusan dan persoalan optimisasi.
- ☐ Saya dapat membedakan polynomial-time dan nonpolynomial-time.
- ☐ Saya dapat membedakan tractable, intractable, decidable, dan undecidable.
- ☐ Saya dapat menjelaskan algoritma deterministik dan non-deterministik.
- ☐ Saya dapat menjelaskan kelas P dan NP tanpa menyamakan NP dengan non-polynomial.
- ☐ Saya dapat menjelaskan $P \subseteq NP$.
- ☐ Saya dapat menjelaskan syarat NP-Complete dan NP-Hard.
- ☐ Saya dapat menentukan arah reduksi polinomial yang benar.
- ☐ Saya dapat menjelaskan konsekuensi jika satu persoalan NPC memiliki algoritma polinomial.
- ☐ Saya dapat mengklasifikasikan contoh umum seperti SAT, Hamiltonian Circuit, TSP decision problem, MST, Euler Circuit, dan Halting Problem.

10.10.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Definisi P, NP, NPC, NP-Hard, $P \subseteq NP$, reduksi polinomial, syarat pembuktian NPC, dan konsekuensi jika NPC berada di P.
Cukup Pahami Konsep	SAT, TSP decision problem, Hamiltonian Circuit, Clique, Euler Circuit, MST, Halting Problem, tractable, intractable, dan undecidable.
Sudah dikuasai	Belum ditentukan.